

Section 1: About You

Goals and Objectives

Why do you want to do a GSoC project with Oppia?	My interest in Oppia began when seniors from my college shared their experiences contributing to the project. Their stories about Oppia's collaborative review culture, emphasis on code quality, and the tangible impact of their work on learners worldwide genuinely resonated with me. Rather than simply following their path, I wanted to understand for myself what makes Oppia special. So, I started contributing. What struck me from my very first PR was how much thought goes into every change. The review process taught me things I never encountered in coursework: writing meaningful tests, structuring code for maintainability, and thinking about how each change fits into the broader system. Debugging CI failures, understanding Bazel build pipelines, and navigating a large-scale Kotlin codebase has sharpened my Android development skills significantly. Beyond the technical growth, Oppia's mission matters to me. Growing up in India, I've seen firsthand how access to quality education varies drastically. The idea that interactive, well-structured lessons can reach anyone with a phone is powerful and knowing that my contributions directly support that mission makes the work feel purposeful. GSoC with Oppia represents a chance to take on a substantial engineering challenge while continuing to learn from a community that genuinely cares about both code quality and educational impact.	
What do you hope to learn/achieve during GSoC?	During GSoC with Oppia, I want to deepen my understanding of CI/CD pipeline, specifically how to architect reliable, multi-stage workflows that handle builds, signing, and deployments with proper security boundaries. I'm also keen to learn infrastructure security patterns like keyless authentication and secret management that are standard in production but rarely covered in coursework. On the development side, I want to improve my ability to write well-tested Kotlin tooling that integrates with Bazel and is robust enough to run unattended in CI. Finally, I hope to strengthen my technical communication skills through the review process thereby writing clear proposals and responding constructively to feedback on complex system design along with gaining hands-on experience in building production infrastructure for a large open-source project..	
Which Oppia teams have you collaborated with, and what have you done on those teams?	Android CLaM	Member of CLaM Team
Contact information	Name: Sandesh Raj Email: sandeshraj0410@gmail.com LinkedIn: https://www.linkedin.com/in/sandesh282/ Github: https://github.com/Sandesh282/	

Preferred method of communication	Email Google Chat
Social media handles (optional) <i>This field is optional. We ask for it so that we can tag you in GSoC-related posts if your application is accepted!</i>	LinkedIn Instagram
Which timezone will you primarily be in during the summer?	Indian Standard Time (+5:30 GMT)
(If you are a student) When are your school holidays?	<i>I am currently a second-year B.Tech student. My summer holidays run from June 8, 2026 to July 13, 2026, during which I can work full-time on GSoC.</i>
What other obligations might you need to work around during the summer?	During this summer, I have no academic obligations or other commitments, so I can work on GSoC full-time.
Planned time commitment	My planned time commitment for GSoC will be structured as follows: • During summer holidays (June 8 – July 13, 2026): 7 - 8 hours/day (approximately 35 - 40 hours/week) • During college (July 14 – September 2026, without exams): 6 - 7 hours/day (approximately 30 - 35 hours/week) • During the College exam period (if applicable): 4 hours/day, ensuring I balance both GSoC and academics properly. Overall, I anticipate dedicating approximately 375+ hours to the project, aligning with the expected workload. Additionally, I am open to working on weekends to meet milestones and ensure timely progress.

Project Timeframe

Coding period	• I will adhere to the above deadlines.
----------------------	---

Communication Channels

I can commit to sending daily updates to the team group chat, each day I work during the GSoC period.	• Yes
In addition to the above: how often, and through which channel(s), do you plan on communicating with your mentor?	I plan to stay in close communication with my mentor throughout the GSoC period: • Weekly Sync Meetings: I'd like to schedule a weekly call via Google Meet (or any preferred platform) to discuss progress, review feedback, and plan upcoming

	work. • Daily Check-ins: I will post brief progress updates via Google Chat or email to maintain transparency and flag blockers early. • Ad- hoc Discussions: For quick questions or clarifications, I'll use Google Chat to keep things moving without waiting for scheduled meetings. This approach ensures steady collaboration, timely feedback, and consistent project momentum.
--	---

Contributor Briefing

If accepted, I will be able to attend the contributor briefing at 3 pm UTC on May 1.	• Yes
--	---

Section 2: About Your Project

Project Details

Project title <i>(should match the one on the Project Ideas list)</i>	4.2 Streamlining Release Automation
Project size <i>(should match the options on the Project Ideas list)</i>	Large (~350 hours)
Why did you choose this project?	I chose the Streamlining Release Automation project because it sits at the intersection of two areas I find genuinely compelling: build systems engineering and developer productivity. Having spent time navigating Oppia's Bazel-based build pipeline, script ecosystem, and Github Actions workflows during my contributions, I've developed a strong appreciation for how much well-designed tooling multiplies a team's effectiveness and a clear-eyed view of where the gaps are. What drew me the most was the scale of the problem. A 17-page manual release process that is both incomplete and outdated means that every release carries unnecessary risk and costs the team hours that could be spent on the product itself. As someone who has debugged CI failures and traced build paths through Bazel rules in this codebase, I understand how tightly coupled these systems are and that's exactly what makes automating them a meaningful engineering challenge rather than a straightforward scripting task. By working on this project, I get to build infrastructure that directly reduces friction for the entire team, from release coordinators to QA testers, while gaining deep experience in areas like cloud-based signing, multi-service deployment

	pipelines, and secure CI/CD design that are difficult to encounter outside of production environments.
--	--

Required Skills (Project-Specific)

GENERAL	
I can figure out the root cause of an issue, and communicate it effectively using a debugging doc .	Debugging Doc for #5886: Slow Android Lint Execution Time
I can debug and fix CI failures / flakes.	Fixes flaky CI failures
I can make changes to GitHub Actions workflows.	I built MergeStation with two Github Actions workflows from scratch: update.yml (cron-based, workflow_dispatch inputs, two-job artifact passing, environment: production protections) and validate.yml (PR Validation with schema checks and dry-run API testing). These demonstrate patterns directly relevant to 4.2 like cron scheduling, manual dispatch, multi-job pipelines, and environment protections.
ANDROID	
Build the app and install it on a local device or emulator. Then verify that you can (a) build a non-test library target locally, (b) run a unit test locally, (c) play through the app locally and (d) build each flavor of the app: dev, alpha, beta, GA.	(a) A link of Github gist (b) Screenshot of running and passing StateFragmentTest (c) Video of running and playing through app locally (d) Link of building each flavor of the app: (i) dev (ii) alpha (iii) beta (iv) ga
Write new Kotlin code with unit tests.	• Fixes part of #4971: Compute aab differences tests • Fix #5818: Robolectric edittext input action • Fix #5998: Fix audio player crashes in AudioPlayerController and AudioViewModel • Fixes part of #5035: Add missing app data reset flow tests
Change Android UIs, write tests for them, and manually verify that they work.	• Fixes part of #5035: Add missing ProfileChooserViewModel data reset tests • Fix #5998: Fix audio player crashes in AudioPlayerController and AudioViewModel

Section 3: Proposal Details

Problem Statement

Target Audience	The primary target audience is Release Coordinators and QA testers on the Oppia Android team. Release coordinators need a reliable, streamlined way to build, sign and deploy releases without following a 17-page manual document. QA testers need a dependable method to receive the correct build version for testers without encountering Play Store synchronization issues.
Core User Need	As a release coordinator, I need an automated way to build, sign and deploy app releases from a single interface, so that I don't have to follow a 17-page manual process that takes multiple hours, risks mistakes from outdated instructions, and must be repeated entirely for every release candidate. As a QA tester, I needed to reliably receive the correct build version for testing, so that I'm not blocked by Play Store synchronization issues that have prevented testers from participating in past releases.

Section 3.1: WHAT

Key User Stories and Tasks that will be implemented

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that"</i>	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
1	Start a Release	As a release coordinator, I need to initiate a new monthly release with minimal manual steps, so that the process is fast, reliable, and less error-prone.	1. Cut a new release branch from the <i>latest-alpha</i> tag following <i>release-<major>.<minor></i> naming.	Tracking Issue (#6106)
			2. Send a PR on develop to bump the minor version in <code>version.bzl</code> . 3. Merging the PR triggers automatic changelog generation via LLM. 4. If changelog generation fails, manually create the changelog. 5. Run <code>build_and_sign.yml</code> for beta and GA flavors; signed AABs are uploaded to GCS archive. 6. Run <code>deploy_to_firebase.yml</code> for beta and GA to push builds to QA. 7. Complete remaining manual steps (email marketing, create release	Version code strategy (#5033)

			tracking doc).	
2	Build, Sign & Archive a Release	As a release coordinator, I need to build, securely sign, and archive release binaries in a single workflow, so that the exact binary tested by QA is the one eventually deployed to end users.	1. Run <code>build_and_sign.yml</code> with inputs: flavor (<code>alpha/beta/ga</code>) and source ref (<code>latest-alpha</code> or <code>release-*</code> branch). 2. Workflow validates inputs, builds AAB with <code>bazel build --compilation_mode=opt</code>. 3. AAB is signed via Cloud KMS using Java's built-in <code>jarsigner</code> with the Cloud KMS provider (key never leaves KMS boundary). 4. Signed AAB is uploaded to the flavor-specific GCS bucket (e.g., <code>oppia-android-beta-releases</code>). Fails if binary already exists. 5. All GCP access uses WIF keyless auth through the <code>oppia-android-release-env</code> GitHub environment (requires reviewer approval).	Cloud KMS HSM docs WIF keyless auth
3	Launch a Release	As a release coordinator, I need to deploy reviewed builds to end users via Play Console with controlled rollout percentages, so that we can catch issues before full rollout.	1. Run <code>deploy_to_play_console.yml</code> for beta (25% rollout); downloads signed AAB from GCS archive. The correct changelog is automatically sourced from <code>config/changelogs/</code> and uploaded alongside the binary.	One-click deployment spec (#5394)
			2. Run <code>deploy_to_play_console.yml</code> to deploy GA AAB to Play Console production track (25% rollout). 3. The changelog (version & flavor) is selected and uploaded with the release. If the changelog doesn't exist, the workflow fails, a changelog must always be included. 4. Wait for Google review and approval. 5. After ~1 week, check app health & crash metrics. 6. Run <code>deploy_to_play_console.yml</code> to update beta to 100%. 7. Run <code>deploy_to_play_console.yml</code> to update GA to 100%.	Play Developer API (Edits)
4	Complete a Release	As a release coordinator, I need to finalize the	1. Verify all Play Console tracks are at 100% rollout. 2. Ensure the next release coordinator is	Tracking issue (#6106)

		release and ensure the team is set up for the next cycle, so that releases run on a predictable monthly cadence.	assigned. 3. Confirm changelog is published on Play Console.	
5	Weekly Coordination Checks	As a release coordinator, I need a clear checklist of weekly tasks, so that alpha deployments, changelog updates, and dependency PRs don't fall through the cracks.	1. Check for alpha releases pending Google approval. 2. Review changelog update PRs generated by automation. 3. Review and merge pinned lesson version PRs. 4. Review and merge TranslateWiki PRs.	Tracking issue (#6106)
6	Automated Alpha Deployment	As a release coordinator, I need alpha builds to be automatically built and deployed weekly, so that testers always have a recent version without manual effort.	1. Weekly cron triggers <code>auto_release_alpha.yml</code> 2. Workflow scans commits on <code>develop</code> vs <code>latest-alpha</code> tag. 3. Finds latest commit with passing CI via GitHub API. 4. Updates <code>latest-alpha</code> tag if a new candidate exists. 5. Kicks off <code>build_and_sign.yml</code> for the alpha flavor. 6. Kicks off both <code>deploy_to_firebase.yml</code> and <code>deploy_to_play_console.yml</code> (100% rollout for alpha). 7. Fails with descriptive error if commits exist but none have passing CI; exits silently if no new commits. 8. Can also be triggered manually via <code>workflow_dispatch</code>.	Tracking issue (#6106) GitHub Actions (Scheduled events)
7	QA Build Access	As a QA tester, I need to reliably receive the correct build version for testing via Firebase App Distribution, so that I can test without Play Store synchronization issues blocking me.	1. Coordinator runs <code>deploy_to_firebase.yml</code> for the target flavor. 2. Workflow downloads signed AAB from GCS archive (same binary that was built and signed earlier). 3. Firebase CLI uploads AAB to App Distribution under the correct tester group. 4. QA receives notification from Firebase with a direct download link. 5. QA installs the correct build directly; no	Firebase App Distribution investigation (#5984) Firebase App Distribution docs

			Play Store sync ambiguity.	
8	Automated Changelog Generation	As a release coordinator, I need changelogs to be auto-generated from merged PRs and fixed issues using an LLM, so that I only need to review and edit rather than write from scratch.	1. Version bump PR merges on <code>develop</code> triggering <code>generate_changelog.yml</code> 2. Script determines the previous version (e.g., if bumped from 0.17→0.18, changelog is generated for 0.17). 3. Collects all commits between the two newest release branch merge bases with <code>develop</code>. 4. Script collects all issue titles/descriptions marked as fixed in commit messages. 5. Vertex AI LLM generates 2-3 user-facing sentences (features first, bug fixes second, ignoring developer-oriented changes). 6. PR is created with changelog file + reference material (commit list, fixed issues) for human review. Changelogs default to <code>config/changelogs/<major.minor>.md</code> and can be overridden per flavor using <code><major.minor>_<flavor>.md</code> 7. Can also be triggered manually via <code>workflow_dispatch</code>. 8. If the LLM call fails (timeout, quota, API error), the workflow still creates the PR with the raw commit list and a <code><!-- LLM generation failed --></code> marker; the coordinator writes the summary manually. This ensures a changelog is always produced regardless of LLM availability.	Automatic changelog computation (#3923) Vertex AI (Gemini API)
9	Changelog Correction & Update	As a release coordinator, I need changelog updates to automatically sync to Play Console when edited, so that end users always see accurate release notes.	Note: The initial changelog is deployed together with the binary in Story 3 (<code>deploy_to_play_console.yml</code>). This story covers only post-deployment corrections. 1. Coordinator edits a changelog file in <code>config/changelogs/</code> and merges to <code>develop</code>. 2. <code>deploy_updated_changelog.yml</code> triggers on push (path filter: <code>config/changelogs/**</code>). This is only run when the changelog is <i>modified</i>, not	Play Developer API

			when it's first generated. - Script audits existing Play Console tracks, checking if the corresponding release is live. - If the release is live and no pending release exists, uploads the updated changelog. - Fails if the release isn't live yet (to avoid race conditions with binary deployment). - Fails if oldest versions of the changelog have changed (cannot change old versions). It is allowed to change a changelog and re-run the deployment script (which will likely need to happen on occasion if a mistake is made in the changelog). - Can also be triggered manually via <code>workflow_dispatch</code>.	
10	Automated Lesson Pinning	As a release coordinator, I need the pinned lesson versions file to be automatically regenerated weekly, so that lesson content stays current without manual script runs.	- Weekly cron runs <code>download_lesson_list</code> script. - Script regenerates <code>pinned_download_list_versions.textproto</code>. - PR is created (or force-pushed to existing branch, similar to TranslateWiki model). - Coordinator reviews and merges. - Can also be triggered manually via <code>workflow_dispatch</code>.	Tracking issue (#6106)
11	Release Documentation	As a release coordinator, I need comprehensive, up-to-date wiki documentation for the release process, so that I can follow the automated playbook and fall back to manual steps if automation fails.	- New wiki page: "App & Feature Release Process" (overview of binary + feature release lifecycle, developer guide for feature flags). - New wiki page: "Release Playbook" (step-by-step instructions leaning heavily on new automation; must also account for the new release process once #5033 is addressed). - New wiki page: "In-Depth Release Reference" (manual fallback for each automation step, required permissions, linked from the playbook for easy jump to manual instructions). - Playbook links to overview (for context) and reference (for fallback), keeping it focused on minimum coordinator steps.	Existing Oppia Android Release documentation

Testing Plan

Note: Since this project is an infrastructure project with no app-level UI changes, "setup" refers to the CI/runner environment configuration rather than a user logging in. The "user" in all cases is a **release coordinator** or **QA tester** interacting with GitHub Actions and Firebase.

Critical user journeys

User Type	User Story Description	Test definition		
		Setup (ends with a user of the correct type getting logged in)	Test steps	Test expectations
RC	Coordinator starts a new release by cutting a branch and bumping the version.	<code>latest-alpha</code> tag exists at commit <code>xyz789</code> . The current <code>MINOR_VERSION</code> in <code>version.bzl</code> is 17.	1. Cut release branch: <code>git checkout -b release-0.17 latest-alpha</code> .	Branch <code>release-0.17</code> is created at commit <code>xyz789</code> .
			2. Send a PR on <code>develop</code> bumping <code>MINOR_VERSION</code> to 18 in <code>version.bzl</code> .	PR is created with only the version file change.
			3. Merge the version bump PR.	<code>generate_changelog.yml</code> is automatically triggered (path filter on <code>version.bzl</code>). Changelog is generated for version 0.17 (the previous version).
			4. Run <code>build_and_sign.yml</code> for <code>beta</code> and <code>ga</code> with source ref <code>release-0.17</code> .	Both builds complete; signed AABs uploaded to their respective GCS buckets.
			5. Run <code>deploy_to_firebase.yml</code> for <code>beta</code> and <code>ga</code> .	QA team receives both builds via Firebase App Distribution.

			6. Attempt <code>git checkout -b release-0.17</code> from a non-existent tag.	Command fails with a clear error.
RC	Build and deploy a release to Firebase for QA testing(<code>deploy_to_firebase.yml</code>).	1. A signed beta AAB exists in the GCS archive from the build step above. 2. Firebase App Distribution is configured with tester group <code>qa-team</code>. 3. WIF credentials are configured in GitHub environment <code>oppia-android-release-env</code>.	1. Navigate to Actions → <code>deploy_to_firebase</code>. 2. Select source ref: <code>release-0.17</code>, flavor: <code>beta</code>. 3. Click "Run workflow".	The workflow derives expected binary name from flavor + source ref commit hash.
			4. Observe the "Download from Archive" step.	Signed AAB is downloaded from the correct GCS bucket path.
			5. Observe the "Deploy" step output.	Firebase CLI uploads the signed AAB to App Distribution under group <code>qa-team</code> . Release notes from the latest changelog are attached.
			6. Check the workflow summary.	Summary includes: version name (<code>0.17-beta</code>), Firebase release link, AAB size, and tester group name.
			7. Run the same workflow with an invalid flavor (e.g., <code>staging</code>).	The workflow fails with a clear input validation error before any build starts.
RC	Deploy a reviewed build to Play Console with staged rollout (<code>deploy_to_play_console.yml</code>).	1. A signed beta AAB exists in the GCS archive.. 2. Play Console API credentials are available via WIF. 3. The <code>beta</code> track	1. Navigate to Actions → <code>deploy_to_play_store</code>. 2. Select flavor: <code>beta</code>, track: <code>beta</code>, source ref: <code>release-0.17</code>,	The workflow starts and authenticates to Play Console API via WIF. Downloads AAB from GCS.

		on Play Console has no pending releases.	rollout: 25. 3. Click "Run workflow".	
			4. Observe the "Upload" step.	A new edit is created. The signed AAB is uploaded to the beta track with the correct changelog.
			5. Observe the "Set Rollout" step.	Rollout fraction is set to 0.25 (25%).
			6. Observe the "Commit" step.	The edit is committed. Workflow summary shows version code, track, and rollout percentage.
			7. After ~1 week, run with same flavor/ref but rollout: 100.	The existing beta track release is updated to 100% rollout. No new AAB is uploaded.
			8. Attempt to deploy version 0.16 when 0.17 is already on track.	Workflow fails: version inversion detected.
QA Tester	Receive and install a build via Firebase App Distribution.	1. A beta build (0.17-beta) has been deployed to Firebase App Distribution. 2. The tester's email is registered in the qa-team group.	1. Check email for Firebase App Distribution notification.	An email is received with the subject referencing "Oppia" and version 0.17-beta.
			2. Click the download link in the email.	The Firebase App Distribution page opens, showing the correct version name and release notes.
			3. Download and install the APK.	The APK installs successfully without errors.

			4. Open the installed app → Settings → About.	The version displayed is 0.17-beta with the correct version code (200).
			5. Repeat with a different tester not in the qa-team group.	The tester does not receive a notification and cannot access the build via the Firebase link.
RC	Weekly alpha cron deploys the latest stable commit(auto_release_alpha.yml) .	1. latest-alpha tag exists at commit abc123 . 2. 5 new commits merged to develop since abc123 . 3. Commit def456 (3rd newest) is the latest with all CI checks passing. 4. Commits 4 and 5 have failing CI.	1. Cron triggers auto_release_alpha at schedule time (or manually via workflow_dispatch for testing).	The workflow starts and queries GitHub API for commit statuses on develop .
			2. Observe the "Find Candidate" step.	The workflow identifies def456 as the deployment candidate. Commits 4 and 5 are skipped with logged reasons (failing CI check names).
			3. Observe the "Update Tag" step.	latest-alpha tag is moved from abc123 to def456 .
			4. Observe the downstream workflows..	build_and_sign.yml is kicked off for alpha. Then both deploy_to_firebase.yml and deploy_to_play_console.yml (100% rollout) are triggered.
			5. Check workflow summary.	Summary shows: selected commit (def456), skipped commits with reasons, Firebase release link, Play Console upload status.

RC	Alpha cron handles no eligible commits gracefully.	1. <code>latest-alpha</code> tag at commit <code>abc123</code>. 2. 3 new commits on <code>develop</code>, all with failing CI.	1. Cron triggers <code>weekly_alpha_deployment</code> .	The workflow starts and queries commit statuses.
			2. Observe the "Find Candidate" step.	No eligible commit found. All 3 commits are listed with their failing CI check names.
			3. Check workflow outcome.	The workflow fails with a descriptive error message. <code>latest-alpha</code> tag remains at <code>abc123</code> . No deployment occurs.
RC	Changelog is auto-generated after the version bump(<code>generate_changelog.yml</code>).	1. A version bump PR (→ <code>MINOR_VERSION = 18</code>) has been merged to <code>develop</code>. 2. Two release branches exist: <code>release-0.16</code> and <code>release-0.17</code>. 3. 15 PRs have been merged between their merge bases. 4. Vertex AI API credentials available via WIF.	1. Observe <code>generate_changelog</code> workflow triggered by the merge event.	The workflow starts and runs <code>GenerateChangeLogs</code> Kotlin script.
			2. Observe the "Collect Commits" step.	The script determines this is for version <code>0.17</code> (the previous version). Collects all 15 merged PRs and their linked issues between <code>release-0.16</code> and <code>release-0.17</code> merge bases.
			3. Observe the "Generate via LLM" step.	The Vertex AI API is called with the commit/issue data. A user-facing changelog summary is generated.
			4. Check <code>develop</code> for a new PR.	A PR is created by adding <code>config/changelogs/0.17.md</code> (changelog file). The PR body includes the raw commit list for reviewer reference.

			5. Verify the changelog content.	The changelog contains 2-3 concise, user-facing sentences summarizing the changes. No internal implementation details are exposed.
			6. Simulate Vertex AI API failure (timeout/error).	The workflow still creates a PR with the raw commit list and a note that LLM generation failed. The coordinator can manually write the changelog.
RC	Changelog update syncs to Play Console (<code>deploy_updated_changelog.yml</code>).	- Changelog <code>config/changelogs/0.17.md</code> edited and pushed to <code>develop</code>. - Version 0.17 beta is live on Play Console. - No pending release exists on the beta track.	1. Observe workflow triggered by changelog path change.	Script audits Play Console tracks, finds 0.17-beta is live.
			2. Observe "Upload Changelog" step.	Updated changelog uploaded to the beta track on Play Console.
			3. Repeat when version 0.18 changelog is edited but 0.18 hasn't been deployed yet.	Workflow fails: "Release 0.18 is not live on Play Console."
			4. Repeat when a pending release exists for the track.	Workflow fails: "Pending release exists, cannot update changelog (race condition risk)."
RC	Pinned lesson versions auto-regenerated weekly (<code>pull_latest_lesson_versions.yml</code>).	- Cron schedule reached. <code>develop</code> branch has existing <code>config/pinned_download_list_versions.txtproto</code>.	1. Cron triggers workflow (or manually via <code>workflow_dispatch</code>).	<code>download_lesson_list</code> script runs and regenerates the textproto file.
			2. Check for PR on <code>develop</code> .	PR created (or force-pushed to existing branch). Diff shows updated lesson IDs/versions.

RC	Coordinator performs weekly release health checks (Weekly Coordination).	Alpha cron has run at least once. Changelog and pinned version PRs may be open.	1. Check Actions tab for latest <code>auto_release_alpha.yml</code> run.	Can verify whether alpha was deployed or skipped (with reason).
			2. Review open PRs for <code>auto/changelog-*</code> branches.	Changelog PRs visible with LLM-generated content and reference links.
			3. Review open PRs for pinned lesson version updates.	Pinned version PR visible with updated textproto diff.
			4. Check for pending Google review approvals on Play Console.	Coordinator can verify approval status for any pending track.
RC	Coordinator uses wiki documentation to follow the release process (Release Documentation).	All wiki pages published under the "Releasing" top-level section.	1. Navigate to "App & Feature Release Process" wiki page.	Page contains overview of binary and feature release lifecycle covering all flavors.
			2. Navigate to "Release Playbook" wiki page.	Step-by-step instructions using new automation workflows. Includes fallback links to in-depth reference. Accounts for post- #5033 process.
			3. Navigate to "In-Depth Release Reference" wiki page.	Manual fallback procedures for every automated step with required permissions.
			4. Follow the playbook to simulate a release start.	Instructions are clear and each step matches actual automation behavior.

Unit Testing (Scripts)

Each of the 6 5 new Kotlin scripts is designed with testable abstractions via an interface + fake pattern. The following table summarizes how each script is unit-tested:

Script	Interface(s) Used	Test Fake(s)	What Unit Tests Verify
SignReleaseBinary	CloudSigner	FakeCloudSigner (returns a predictable signature)	Correct AAB digest extraction, signature validation logic, error handling (KMS unreachable, key disabled, cert mismatch, missing AAB)
TransformAndroidManifest	GitClient, CommandExecutor	FakeCommandExecutor	Version code computation from commit count, version name construction following <code><major>.<minor>-rc<num>-<flavor>-<hash></code> pattern, manifest attribute injection
UploadBinaryToPlayConsole	PlayConsoleClient	FakePlayConsoleClient (simulates track states, pending releases, version codes)	All 4 precondition checks (changelog exists, no pending release, no version regression, no version code inversion), correct Edit → Upload → SetTrack → Commit sequence, rollout percentage handling

UploadChangelogToPlayConsole	PlayConsoleClient	FakePlayConsoleClient	Track auditing logic, changelog diff detection, correct upload to live tracks, error cases (pending release exists, changelog version newer than live release)
GenerateChangelogs	VertexAiClient, GitHubClient	FakeVertexAiClient (returns canned LLM text), FakeGitHubClient (returns test commit/issue data)	Commit collection between release branches, issue parsing from Fixes #NNNN, LLM prompt construction, PR creation with reference material, graceful degradation when LLM fails (raw commit list + failure marker)
FindAlphaCandidate	GitHubClient	FakeGitHubClient (returns test commit statuses)	Candidate selection (newest commit with all CI passing), correct handling of no new commits (exit 0), no eligible commits (exit 1 with reasons), GitHub API retry logic

Key point: All unit tests run via `bazel test` on both macOS (local development) and Ubuntu (GitHub Actions CI) using only the existing Bazel build infrastructure. No GCP credentials, no Firebase project, no Play Console access, and no GitHub tokens are required for unit tests. The interface + fake pattern ensures complete isolation from external services.

All test files follow the existing codebase convention and are placed under `scripts/src/javatests/org/oppia/android/scripts/release/`, using JUnit + Truth assertions.

Integration Testing (Workflows & Real Service)

Integration testing verifies that the GitHub Actions workflows correctly orchestrate the Kotlin scripts and interact with real GCP services. This layer of testing uses a personal GCP project (described in Section 3.2, Infrastructure Setup) to avoid any dependency on Oppia's production infrastructure.

Workflow	Integration Test Approach
<code>build_and_sign.yml</code>	Trigger manually via <code>workflow_dispatch</code> on a personal fork. Verify: Bazel build succeeds, <code>SignReleaseBinary</code> signs the AAB against a test Cloud KMS key, and the signed AAB is uploaded to a test GCS bucket.
<code>deploy_to_firebase.yml</code>	Trigger manually. Verify: AAB is downloaded from the test GCS bucket, Firebase CLI uploads to a test Firebase App Distribution project, testers in the test group receive the build notification.
<code>deploy_to_play_console.yml</code>	Trigger manually. Verify: AAB is downloaded from the test GCS bucket, <code>UploadBinaryToPlayConsole</code> creates an Edit, uploads the AAB, sets the correct rollout percentage, and commits the Edit against a test Play Console app.
<code>deploy_updated_changelog.yml</code>	Trigger via a push to <code>config/changelogs/</code> on a test branch. Verify: <code>UploadChangelogToPlayConsole</code> detects the changelog diff and uploads to the correct live track on the test Play Console app.
<code>generate_changelog.yml</code>	Trigger via a push to <code>version.bzl</code> on a test branch. Verify: <code>GenerateChangelogs</code> collects commits, calls Vertex AI, and creates a PR with the generated changelog. Also test the LLM failure fallback by temporarily blocking API access.
<code>auto_release_alpha.yml</code>	Trigger manually via <code>workflow_dispatch</code> . Verify: <code>FindAlphaCandidate</code> selects the correct commit, tag is updated, and downstream workflows (<code>build_and_sign</code> , <code>deploy_to_firebase</code> , <code>deploy_to_play_console</code>) are triggered in sequence.
<code>pull_latest_lesson_versions.yml</code>	Trigger via <code>workflow_dispatch</code> . Verify: <code>download_lesson_list</code> script runs, textproto file is regenerated, and a PR is created/updated on the <code>automated/lesson-versions</code> branch.

End-to-End Acceptance Testing

End-to-end acceptance tests validate the complete release lifecycle from the **release coordinator's perspective** – matching the Critical User Journeys in the user stories above.

The acceptance tests are performed against Oppia's actual infrastructure after the required GCP/GitHub/Firebase/Play Console setup is complete:

- **M1 acceptance tests** (User Stories 1–4, 7, 9): Verified after M1 Row 10 is merged, demonstrated at the PM Demo (M1 Row 11), and documented in M1 Row 12. These cover: starting a release, building + signing + archiving, deploying to Firebase, deploying to Play Console, and changelog updates.
- **M2 acceptance tests** (User Stories 5, 6, 8, 10, 11): Verified after M2 Row 6 is merged, demonstrated at Mentor Demos (M2 Rows 3 and 7), and documented in M2 Row 12. These cover: weekly coordination, automated alpha deployment, changelog generation, lesson pinning, and release documentation.

Documentation Testing

The 4 new wiki pages will be validated by:

1. **Self-review**: Following each playbook step manually to verify correctness.
2. **Mentor review**: Mentors review the wiki page PRs for clarity, accuracy, and completeness.
3. **Cross-reference**: Ensuring every automated workflow step referenced in the Playbook has a corresponding manual fallback documented in the In-Depth Release Reference.

Technical Requirements

Data Handling and Privacy

#	Type of data	Description	Why do we need to store this data?	Anonymized?	Can the user opt out?	Wipeout policy	Takeout policy
1.	Cloud KMS signing keys	Release signing keys stored in Google Cloud KMS (managed HSM). Keys never leave the KMS boundary; only the signature is returned.	Required to sign AABs for Play Store deployment.	N/A (not user data. Keys are org-level infrastructure).	N/A	N/A (not tied to any user account.)	N/A
2.	Firebase App Distribution	Tester emails managed via a Google Group in	Required to deliver QA builds to the correct testers.	No (emails are needed for delivery.)	Yes (testers can be removed from the Google	Removed when the tester leaves	N/A (managed by

	tester emails	Firebase console. Not stored in the Oppia codebase.			Group.)	the group.	Firebase, not Oppia's datastore.)
3.	Changelog data	Generated from public GitHub commit history and issue titles.	Required for Play Store release notes.	Yes (contains only public commit messages, no user PII.)	N/A	N/A (not tied to any user.)	N/A
4.	Release binaries (GCS)	Signed AABs stored in Google Cloud Storage buckets per flavor.	Required for reproducible deployments; same binary tested by QA is deployed to end users.	N/A (not user data.)	N/A	Alpha: auto-deleted after 90 days. Beta: archived after 90 days, deleted after 365 days. Prod: archived after 90 days, never deleted.	N/A

Connection to Existing Work

The following existing codebase components serve as direct foundations for this project:

Existing Component	Location	How 4.2 Builds On It
AAB build pipeline	build_flavors.bzl , oppia_android_application.bzl	Reused as-is to produce <code>oppia_{flavor}.aab</code> release artifacts in CI workflows.
Version constants	version.bzl	The release coordinator manually bumps the version in this file via a PR. The existing Bazel propagation chain (<code>version.bzl</code> → <code>build_flavors.bzl</code> → <code>TransformAndroidManifest</code>

		<code>.kt</code> → final AAB) handles the rest automatically.
Manifest transform	TransformAndroidManifest.kt	Already injects version code and version name at build time. No changes needed.
Git utilities	GitClient.kt	Reused for branch operations, commit comparison, and tag management in release scripts.
GitHub API client	GitHubClient.kt	Extended to support PR creation, release tagging, and CI status queries via the GitHub REST API.
Command executor	CommandExecutorImpl.kt	Reused for all shell command execution (Bazel builds, signing tools, Firebase CLI, <code>gcloud</code> commands).
Script patterns	All scripts in <code>scripts/src/java/org/oppia/android/scripts/</code>	New scripts follow the established pattern: <code>main()</code> entry point, argument parsing, Bazel <code>kt_jvm_binary</code> build targets, textproto configurations.
Existing CI workflows	.github/workflows/	New workflows follow existing patterns for Bazel setup, caching, and runner configuration. The set-up-android-bazel-build-environment composite action is reused in all build workflows.

New patterns introduced: This project introduces **no new architectural patterns** to the Android app code. For CI/CD, the following new-to-Oppia patterns are introduced:

- **Workload Identity Federation (WIF)** for keyless authentication to GCP services (replaces stored service account keys).
- **GitHub Environments with required reviewers** (`oppia-android-release-env`) to gate release workflows behind manual approval, locked to `release-*` branches and the `latest-alpha` tag.

- **Reusable GitHub Actions workflows** ([workflow_call](#)) to share build-sign-deploy logic across deployment targets.
- **Cloud KMS remote signing** via the [SignReleaseBinary](#) custom Kotlin script with an HSM-backed keystore (key never leaves KMS boundary).
- **Google Cloud Storage binary archive** for reproducible deployments: build once, deploy the same binary to QA and production.
- **LLM-assisted changelog generation** using the Vertex AI API to summarize commit history into user-facing release notes.

Other Requirements

1. **Security:** All signing keys must remain in Google Cloud KMS and never be exposed to CI runners. GitHub Actions workflows authenticate to GCP via Workload Identity Federation (WIF), which uses short-lived OIDC tokens instead of stored service account keys. The [oppia-android-release-env](#) GitHub environment enforces required reviewer approval and restricts deployments to [release-*](#) branches and the [latest-alpha](#) tag. WIF attribute conditions provide defense-in-depth: `attribute.repository == "oppia/oppia-android"` AND `attribute.environment == "oppia-android-release-env"` AND matching ref pattern.
2. **Platform compatibility:** All Kotlin scripts must function identically on macOS (developer machines) and Ubuntu (GitHub Actions runners). This is ensured by using Bazel as the build/run system and avoiding OS-specific shell commands.
3. **Backward compatibility:** All new workflows run alongside the existing manual release process. No existing scripts, workflows, or release steps are removed or modified until the automated replacement has been tested end-to-end and approved by the release coordinator.
4. **Third-party libraries:** The project expects to use existing libraries for Play Store and Firebase integration, as specified in the [project description](#). Specific libraries will be evaluated during community bonding:
 - **Play Store:** Direct use of the [Google Play Developer API client library](#). Gradle-based wrappers like [play-publisher](#) are incompatible with Bazel.
 - **Firebase:** Firebase CLI (`firebase appdistribution:distribute`) or the [Firebase App Distribution REST API](#) for build uploads.
 - **Signing:** Custom Kotlin script ([SignReleaseBinary](#)) using Cloud KMS HSM-backed keystore. The public PEM certificate (X.509) is checked into `config/certificate/` in plaintext.
 - **LLM:** Vertex AI Gemini API via REST (no additional client library required for simple text generation). The GitHub Releases API can also produce auto-generated release notes from tag diffs, which may serve as supplementary input alongside the richer commit and issue data the [GenerateChangelogs](#) script already collects.
 - **Cloud storage:** `gcloud storage cp` via [google-github-actions/setup-gcloud](#) for downloading archived binaries (no existing download action available).

5. **Version code strategy:** [#5033](#) is planned to be implemented before GSoC coding begins, so the computed version code strategy can be assumed to be present. No additional work is expected in this project for updating version codes.
 6. **Binary archive structure:** Signed AABs are stored in flavor-specific GCS buckets following the path `gs://<bucket>/<major>.<minor>/RC<num>/<name>.aab`. Example: `gs://oppia-android-beta-releases/0.16/RC01/oppia-android-0.16-rc01-beta-5f3e75afa7.aab`. Retention policies: alpha (90-day auto-delete), beta (90-day archive, 365-day delete), prod (90-day archive, never delete).
 7. **Scope exclusions:** Per the [project description](#), the following are explicitly **out of scope**:
 - Cron-based automation for beta and GA launches (only alpha crons are in scope)
 - Changelog translations (only English generation)
 - GitHub release creation
 - Marketing emails and GitHub Discussions posts
-

Section 3.2: HOW

Existing Status Quo

The current release process is governed by a [17-page private Google document](#) that is both incomplete (multiple sections are *TODO(Ben)*) and partially outdated. The process requires the release coordinator to:

1. **Set up a dedicated local environment** — cloning three repositories (oppia-android, a redacted asset repository, and a redacted key repository), installing specific SDK versions (19 and 28) and build tools (29.0.2, 31.0.0), and clearing Bazel caches.
2. **Manually prepare the binary** — swapping module configurations (e.g., replacing `DeveloperOptionsStarterModule` → `LogReportingModule`, toggling `CachingModule` flags, editing `ApplicationComponent.kt`, modifying two `BUILD.bazel` files), downloading production assets via a separate script, and copying them into `domain/src/main/assets`.
3. **Build, sign, and archive locally** — running `bazel build --compilation_mode=opt`, signing each AAB with a local keystore via `apksigner`, creating universal APKs with `bundletool`, and manually archiving signed binaries into the key repository (which uses Git LFS that is reportedly broken).
4. **Deploy via Play Console UI** — manually uploading AABs to internal testing, promoting to closed testing, and uploading proguard mapping files through the Play Store's web interface with no scripted support.
5. **Repeat for each release candidate** — every bug fix requires cherry-picking, rebuilding, re-signing, and re-uploading. The entire Build → Sign → Upload cycle takes multiple hours per iteration.

Key pain points the current process creates:

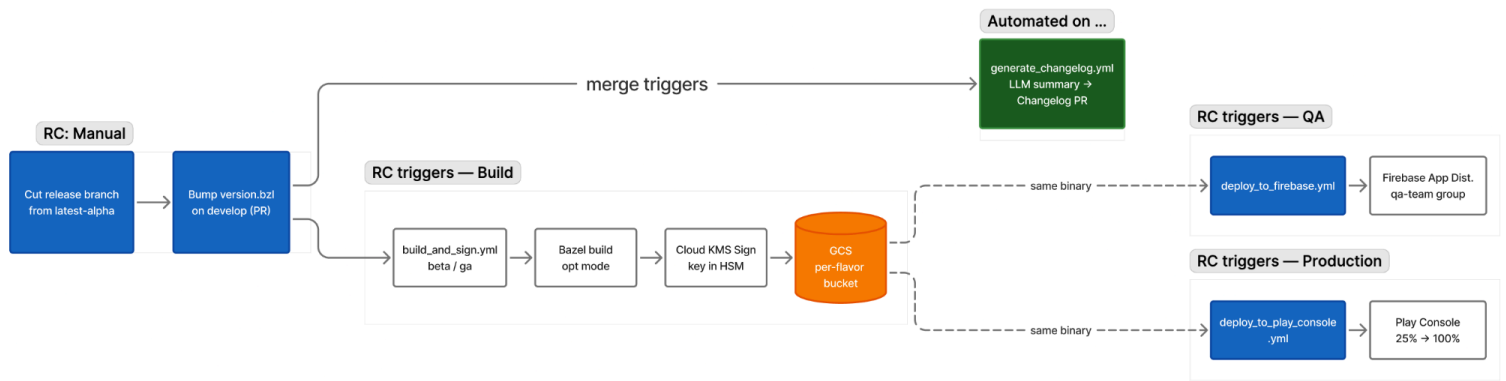
- **No automation exists.** Every release is an entirely manual, multi-hour process with 22+ documented steps (and several undocumented ones).
- **Unreliable QA build distribution channel.** QA testers rely on Play Store Internal Testing, which has a [well-known synchronization issue](#) making it difficult to guarantee testers receive the correct build. This has caused testers to be unable to participate in past releases.
- **Signing keys live locally.** The keystore is decrypted (`git secret reveal`) on the coordinator's machine, creating a single point of failure and security risk.
- **No changelog automation.** Release notes are written entirely from scratch by the coordinator.
- **Lesson version pinning risks build reproducibility.** Pinned lesson versions are determined at release time, and the pinned version file has been accidentally lost when creating new RCs in the past. Without automation to regenerate and track this file, there is a risk of shipping release candidates with inconsistent lesson content.
- **Alpha testers may go weeks without a new build.** Without a scheduled build-and-deploy cycle, alpha releases only happen when a coordinator manually triggers one, delaying feedback loops and reducing the window for catching regressions early.
- **New release coordinators have no reliable onboarding path.** The only process reference is a private Google Doc that is inaccessible to most contributors and contains multiple incomplete `TODO` sections, increasing the risk of mistakes during coordinator handoffs.

Solution Overview

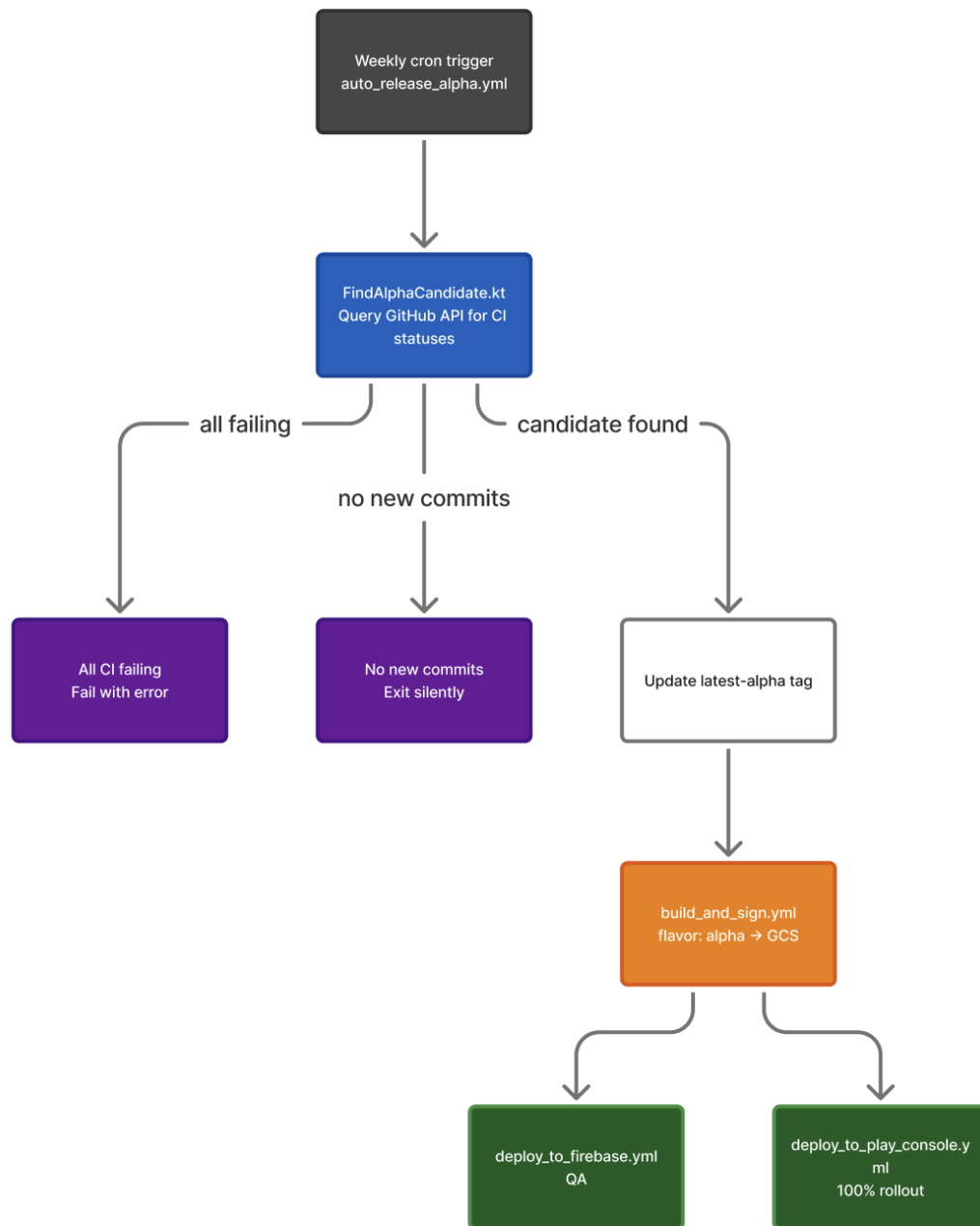
This project replaces the manual release pipeline with a set of **custom Kotlin scripts** and **GitHub Actions workflows** that automate the build-sign-archive-deploy lifecycle end-to-end. The design follows three principles:

1. **Build once, deploy everywhere.** A signed AAB is built exactly once and uploaded to a flavor-specific GCS bucket. The same binary is then deployed to Firebase (QA) and Play Console (production), eliminating binary divergence.
2. **Keyless, zero-trust security.** Signing keys never leave Google Cloud KMS. All GCP access uses [Workload Identity Federation \(WIF\)](#) with short-lived OIDC tokens. Release workflows are gated behind a [GitHub Environment with required reviewers](#).
3. **Scripts follow existing patterns.** All new Kotlin scripts follow the established codebase pattern: `main()` entry point, argument parsing, Bazel `kt_jvm_binary` targets, and existing utilities like [GitClient.kt](#), [GitHubClient.kt](#), and [CommandExecutorImpl.kt](#).

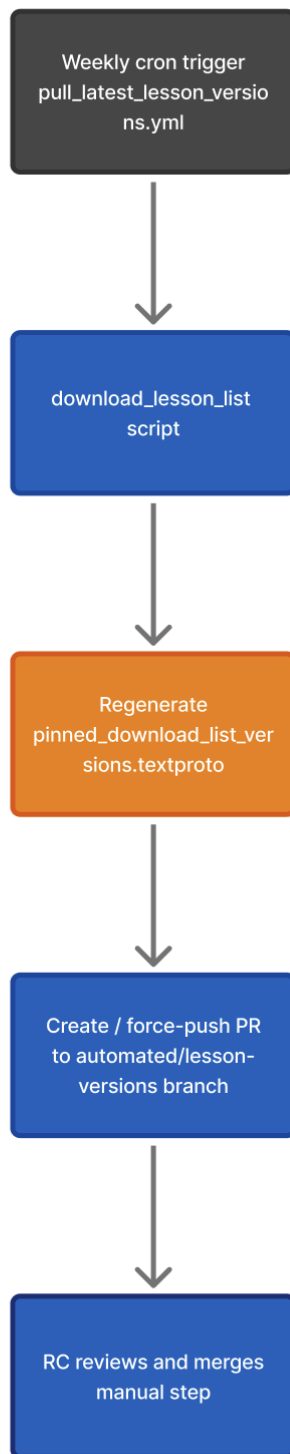
At a high level, the automated release flow becomes:



Monthly Release Flow



Weekly Alpha Flow



Weekly Lesson Pinning

Note on branch cutting: Creating a release branch (`git checkout -b release-X.Y latest-alpha`) and bumping `version.bzl` on `develop` remain manual coordinator steps — they are ~3 commands and do not warrant a dedicated workflow. These steps are documented in the Release Playbook wiki page. Bumping `version.bzl` automatically triggers `generate_changelog.yml`.

Release Lifecycle: From Trigger to Users

Here is the exact data flow when a release coordinator deploys a beta release. Each step shows what happens, where data flows, and why it works that way.

1. **Coordinator triggers `build_and_sign.yml`** from the GitHub Actions UI with inputs: `flavor=beta, source_ref=release-0.17`.
2. **GitHub checks the environment gate.** The workflow specifies `environment: oppia-android-release-env`, so GitHub pauses execution until a required reviewer (org admin) approves. This prevents unauthorized builds even if someone forks the repo, they cannot trigger the environment.
3. **Bazel builds the optimized AAB.** The workflow runs `bazel build --compilation_mode=opt //:oppia_beta` on an Ubuntu runner using the existing [set-up-android-bazel-build-environment](#) composite action. Output: an unsigned `oppia_beta.aab`.
4. **WIF obtains short-lived GCP credentials.** The `google-github-actions/auth` action exchanges the runner's OIDC token for temporary GCP credentials. The WIF identity pool validates three conditions before granting access: `attribute.repository == "oppia/oppia-android"`, `attribute.environment == "oppia-android-release-env"`, and `attribute.ref` matching the `release-*` pattern. This is keyless authentication; no service account JSON key is stored anywhere.
5. **SignReleaseBinary signs the AAB remotely.** The script sends only the AAB's digest to Cloud KMS via Java's built-in jarsigner configured with the Cloud KMS keystore provider. KMS computes the signature using the HSM-backed private key and returns it. The private key never leaves the HSM boundary; this is why we chose Cloud KMS over local keystore decryption in CI (which would expose the key in runner memory). The script then validates the signature by verifying the certificate in `META-INF/`.
6. **TransformAndroidManifest bakes the version name into the AAB at build time.** The version code and name are computed from the git commit count and embedded directly into the binary.
7. **Signed AAB is uploaded to GCS.** `gcloud storage cp` uploads to `gs://oppia-android-beta-releases/0.17/RC01/oppia-android-0.17-rc01-beta-5f3e75afa7.aab`. The upload fails if the file already exists (prevents accidental overwrites). From this point forward, every deployment for this release pulls from this exact archived binary; the build-once-deploy-everywhere guarantee.
8. **Coordinator triggers `deploy_to_firebase.yml`.** The workflow downloads the archived AAB from GCS (fails if not found; the build step must complete first), then runs `firebase appdistribution:distripute` to push it to the `qa-team` tester group. QA testers receive a direct download link via email within minutes; no Play Store sync delay. This is why we chose Firebase over Play Store Internal Testing: it resolves the [sync issue \(#5984\)](#) that blocked testers in past releases.
9. **QA tests the build.** If bugs are found, the coordinator cherry-picks fixes into `release-0.17`, then re-runs `build_and_sign.yml` with the same `source_ref=release-0.17` (which now points to the latest commit on that branch). The new build gets `RC02` in its name. QA re-tests via Firebase.

10. **Coordinator triggers** `deploy_to_play_console.yml`** with `gcs_aab_path` (copied from the GCS upload step above), `track=beta`, and `rollout_fraction=0.25`.
`UploadBinaryToPlayConsole` runs precondition checks (`pending release?` changelog exists in `config/changelogs/0.17.md`), uploads the AAB, then runs the version inversion check against the returned version code before committing. The changelog is sourced from `config/changelogs/0.17.md` (or `0.17_beta.md` if a flavor-specific override exists).
11. **Coordinator increases rollout**. After monitoring crash rates for 24-48 hours, re-runs `deploy_to_play_console.yml` with `rollout_percentage=100`. Same binary, same changelog; only the rollout fraction changes.

For alpha releases, this entire lifecycle is automated by the weekly `auto_release_alpha.yml` cron — steps 1-8 happen without human intervention. `FindAlphaCandidate` selects the latest commit on `develop` with passing CI, updates the `latest-alpha` tag, and chains `build_and_sign` → `deploy_to_firebase` → `deploy_to_play_console` automatically.

Integration with Existing Codebase

Category	Files / Systems	Notes
Reuses (no changes)	<code>scripts/.../common/GitClient.kt</code>	Git operations (branch listing, merge bases, tag resolution)
	<code>scripts/.../common/GitHubClient.kt</code>	GitHub API calls (PR creation, issue queries, CI status checks)
	<code>scripts/.../common/CommandExecutorImpl.kt</code>	Shell command execution wrapper
	<code>.github/actions/set-up-android-bazel-build-environment</code>	Composite action for Bazel + Android SDK setup in CI
	<code>version.bzl</code> (read-only)	Source of truth for major/minor version; new scripts read but never write it
Extends	<code>.github/workflows/</code>	7 new workflow files added alongside existing 12 workflows

	<code>scripts/src/java/org/oppia/android/scripts/</code>	New <code>release/</code> subdirectory with 6 4 scripts, following existing directory conventions
	<code>config/</code>	New <code>changelogs/</code> and <code>certificate/</code> subdirectories
Does NOT modify	Android app source code (<code>app/</code> , <code>domain/</code> , <code>utility/</code> , etc.)	Zero runtime changes — all changes are CI/scripts/docs
	Existing GitHub Actions workflows	No changes to <code>static_checks.yml</code> , <code>unit_tests.yml</code> , etc.
	Bazel build rules for the app	Existing <code>//:oppia_<flavor></code> targets are used as-is

Third-Party Libraries

No.	Third-party library name and version	Link to third-party library	Why it is needed	License (if third-party library)	[Android only] Min / target / max SDK version that the library supports
1.	Google Play Developer API Client Library (v3-rev20260318-2.0.0)	google-api-services-androidpublisher	Programmatic upload of AABs and changelogs to Play Console tracks; querying track status and rollout percentages. Gradle wrappers like <code>play-publisher</code> are incompatible with Bazel.	Apache 2.0	N/A (JVM-only, runs in CI)
2	Firebase CLI v13.29.1	firebase-tools	Upload signed AABs to Firebase App Distribution and manage tester groups. Used via <code>firebase appdistribution:distribute</code> command.	MIT (for npm package)	N/A (CLI tool, runs in CI)

3	google-auth-library-java 1.43.0	google-auth-library-java	Provides <code>ExternalAccountCredentials</code> for WIF token exchange. Needed for authenticating to GCP services (KMS, GCS, Play Console, Vertex AI) from GitHub Actions.	BSD-3-Clause	N/A (JVM-only, runs in CI)
---	------------------------------------	--	---	--------------	----------------------------

Note: No new libraries are added to the Android app itself. All additions above are JVM-only dependencies used exclusively in CI scripts built with Bazel `kt_jvm_binary` targets.

“Service” Dependencies

No.	Dependency name	Why it is needed	What our plan is, if the dependency fails under us
1	Google Cloud KMS (HSM)	Stores the release signing key; signs AABs remotely so the key never leaves the HSM boundary	Fallback: The existing manual signing process using a locally decrypted keystore can be used as a temporary fallback. The manual instructions are documented in the "In-Depth Release Reference" wiki page.
2	Google Cloud Storage (GCS)	Archives signed AABs per flavor with lifecycle policies (alpha: 90-day delete, beta: 365-day delete, prod: never delete).	Fallback: The coordinator can build and sign locally per the existing process documented in the "In-Depth Release Reference" wiki page.
3	Google Play Developer API	Uploads AABs and changelogs to Play Console tracks; manages rollout percentages programmatically.	Fallback: The coordinator manually uploads the AAB through the Play Console web interface, following the documented manual fallback steps.
4	Firebase App Distribution	Distributes QA builds directly to testers, bypassing Play Store sync issues.	Fallback: QA builds can be shared via the Play Store Internal Testing track (the current process) or distributed as direct APK downloads from GCS.
5	Vertex AI (Gemini API)	Generates user-facing changelog summaries from commit history and issue data.	Fallback: If the API call fails, the workflow still creates the changelog PR with the raw commit list and a note that LLM generation failed. The coordinator writes the summary manually. This fallback is explicitly handled in the script logic.

6	GitHub Actions	Hosts all CI/CD workflows for building, signing, deploying, and cron-based automation.	Fallback: Every automated step has a corresponding manual procedure documented in the "In-Depth Release Reference" wiki page. The coordinator can perform any step locally.
---	----------------	--	--

Key High-Level and Architectural Decisions

Decision 1: Cloud KMS Remote Signing vs. Local Keystore Signing

We have considered the following options:

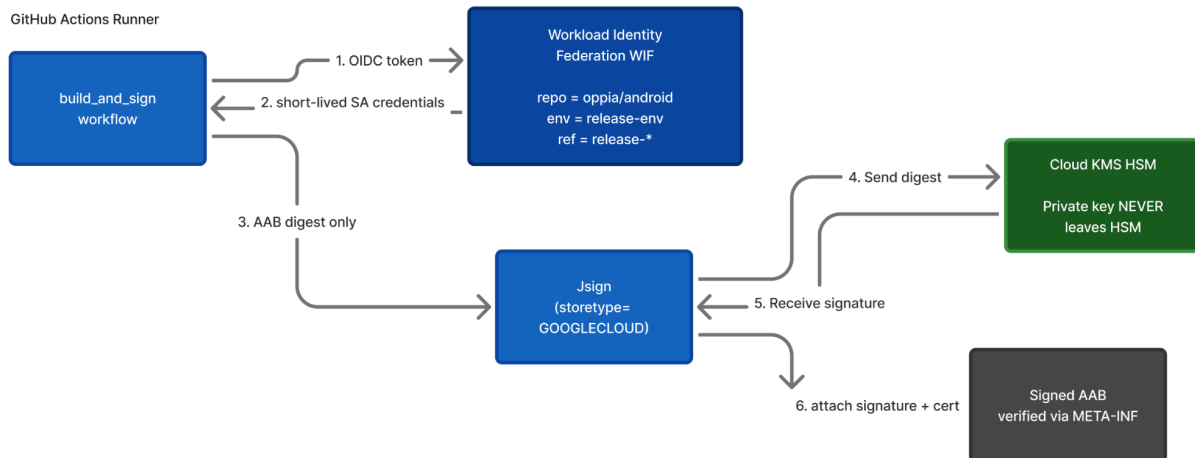
1. **Cloud KMS HSM-backed signing:** The signing key is stored in Google Cloud KMS. Java's built-in `jarsigner` sends the AAB digest to KMS via the Cloud KMS keystore provider, which returns the signature. The key never leaves the HSM.

Note: We already use Play App Signing for the final upload key; this option handles the **upload certificate** signing that authenticates the AAB before Play processes it.
2. **Encrypted keystore in the public repo via `git secret`:** Store the keystore encrypted in the public repository using `git secret`, which requires a service account with a GPG key in GitHub Secrets or a GitHub Environment. Decrypt at workflow time and sign using `apksigner`.
3. **Signing in the private repository:** Run the signing workflow in the private repository that currently holds the key, using a cross-repository workflow handoff (public repo triggers private repo workflow, private repo signs and returns the artifact).

Among these, we believe that **Cloud KMS HSM-backed signing** is the best approach, because:

- The signing key never exists in plaintext on any CI runner, eliminating the risk of key exfiltration from workflow logs, memory dumps, or compromised runners.
- WIF + GitHub Environments provide defense-in-depth: even if a workflow is compromised, the attacker cannot sign without satisfying all WIF attribute conditions AND environment reviewer approval.
- Option 2 (encrypted keystore) still results in the key being decrypted in runner memory, so the distinction from storing the key directly is minimal – but we still need manual access to the key, making this approach fragile.
- Option 3 (private repo signing) introduces a complex public↔private workflow handoff that is difficult to debug and maintain.

Signing & security data flow:



The above approaches are contrasted in detail in the following table:

Criteria	Option 1 (Cloud KMS HSM-backed signing)	Option 2 (Secret Manager Keystore)	Option 3 (Private repo Signing)
Key leaves secure boundary?	No (only digest sent, signature returned)	Yes (decrypted in runner memory)	No (key stays in private repo)
Supports Firebase / direct distribution?	Yes (team controls signing)	Yes (team controls signing)	Yes (team controls signing)
Complexity	Low (single workflow, standard tooling)	Medium (GPG key management, decryption step)	High (cross-repo workflow handoff, artifact transfer)
Access control granularity	WIF conditions + IAM roles + Environment approval	GitHub secret or Environment access	Private repo access controls

Decision 2: Workload Identity Federation vs. Service Account Keys

We have considered the following options:

1. **WIF (keyless auth):** GitHub Actions exchanges an OIDC token for short-lived GCP credentials. No long-lived secrets are stored.
2. **Service account JSON keys:** Store JSON key files as GitHub Actions secrets.

Among these, we believe that **WIF** is the best approach, because:

- No long-lived credentials to rotate, leak, or manage.
- Attribute conditions restrict access by repository, environment, and branch pattern – providing fine-grained, auditable access control.
- This is [Google's recommended approach](#) for GitHub Actions → GCP authentication.

The above approaches are contrasted in detail in the following table:

Criteria	Alternative 1 (WIF- Keyless Auth)	Alternative 2 (Service Account JSON Keys)
Long-lived credentials stored?	No (OIDC tokens are ephemeral)	Yes (JSON key file in GitHub Secrets)
Key rotation required?	No (tokens auto-expire)	Yes (manual rotation recommended every 90 days)
Access control granularity	Attribute conditions: repo, environment, branch pattern	Any workflow in the repo can access the secret
Auditability	Every token exchange logged in GCP Cloud Audit Logs	Secret access not logged by default
Google recommendation	Yes (recommended for GitHub Actions)	Legacy approach, not recommended for new setups

Decision 3: Firebase App Distribution vs. Play Store Internal Testing for QA

We have considered the following options:

1. **Firebase App Distribution:** Upload builds via Firebase CLI; testers receive direct download links via email.
2. **Play Store Internal Testing:** The current approach; testers install via the Play Store.

Among these, we believe that **Firebase App Distribution** is the best approach, because:

- It eliminates the well-known Play Store synchronization issue ([#5984](#)) that has blocked QA testers in past releases.
- Testers receive a direct download link – no Play Store cache, version propagation delay, or opt-in confusion.
- Multiple flavors (alpha, beta, GA) can be distributed simultaneously to different tester groups.
- The exact same signed binary from the GCS archive is uploaded, maintaining binary integrity.

The above approaches are contrasted in detail in the following table:

Criteria	Firebase App Distribution	Play Store Internal Testing
Build sync reliability	Direct download link (immediate)	Subject to Play Store propagation delays (#5984)
Tester onboarding	Email invite (no Play Store opt-in required)	Requires Play Store opt-in to testing track
Multiple flavors simultaneously	Yes (separate tester groups per flavor)	Requires separate track configuration per flavor
Binary source	Same signed AAB from GCS archive	Same signed AAB from GCS archive
Version confusion risk	None (tester downloads exactly the linked version)	High (Play Store may serve cached/stale version)

Decision 4: Separate Build and Deploy Workflows vs. Single Workflow

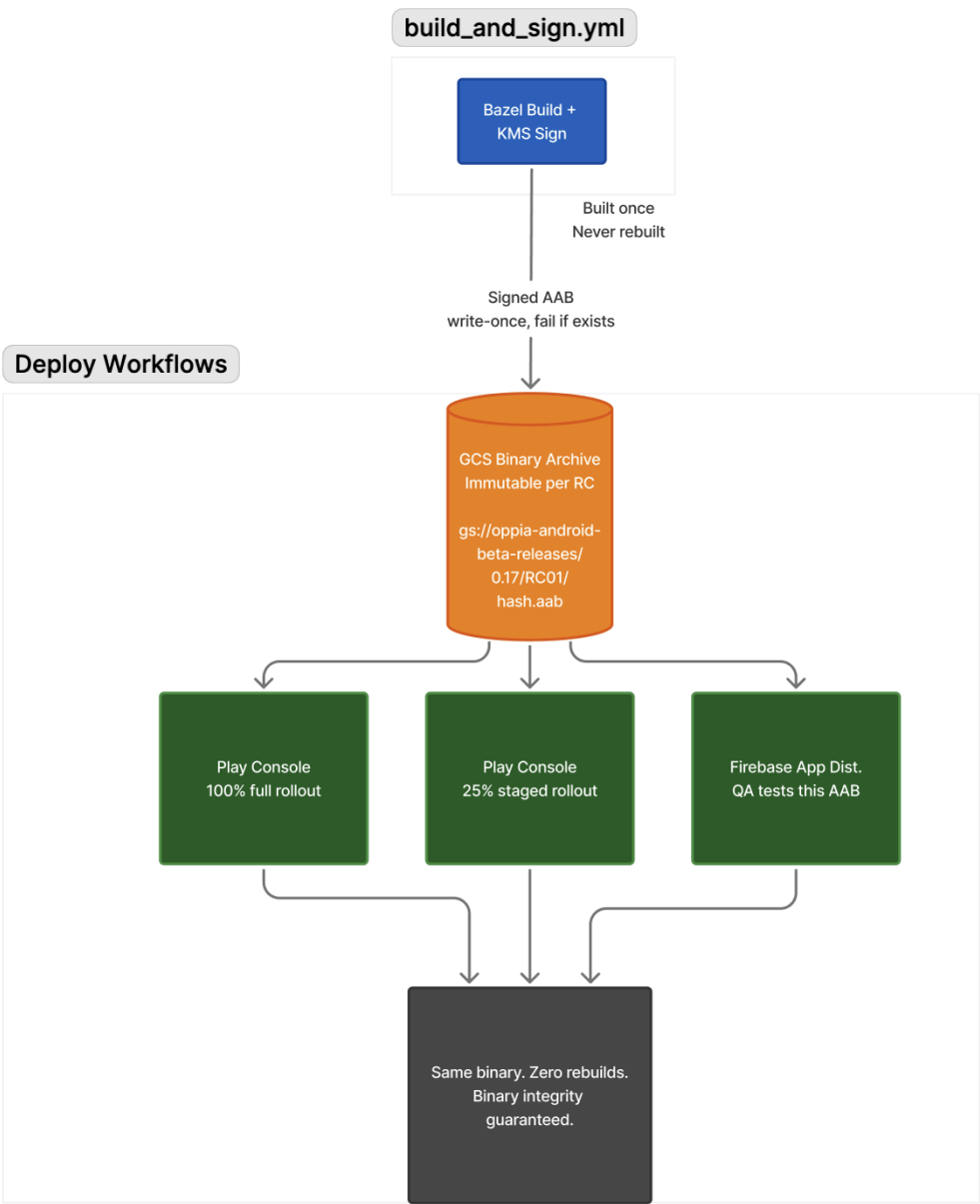
We have considered the following options:

1. **Separate workflows** — `build_and_sign.yml` produces the signed binary and archives it. Separate `deploy_to_firebase.yml` and `deploy_to_play_console.yml` workflows download the archived binary and deploy it.
2. **Single workflow** — One workflow builds, signs, and deploys to all targets in sequence.

Among these, we believe that **separate workflows** is the best approach, because:

- **Binary reproducibility:** The exact AAB tested by QA via Firebase is the same binary later deployed to Play Console. No rebuild required.
- **Failure isolation:** A Firebase deployment failure doesn't block Play Console deployment, and vice versa.
- **Flexible re-deployment:** If a deployment fails (e.g., network timeout), the coordinator can re-run only the deploy workflow without rebuilding. This is the explicitly stated design in the project description.

Build-once-deploy-everywhere flow:



The above approaches are contrasted in detail in the following table:

Criteria	Alternative 1 (Separate Build + Deploy Workflows)	Alternative 2 (Single Monolithic Workflow)
Binary rebuilt per target?	No (built once, archived in GCS, pulled by each deploy)	Yes (each deploy step may trigger a rebuild)

Failure isolation	Firebase failure doesn't block Play Console deployment	Any step failure blocks all subsequent steps
Re-deployability	Re-run only the failed deploy workflow	Must re-run entire workflow from scratch
Binary integrity	Guaranteed; same archived AAB goes to QA and production	Risk of divergent binaries if rebuilt
Rollout flexibility	Coordinator can deploy to Firebase first, Play Console days later	All deployments happen in same run

Risks and mitigations

Potential Risk	Mitigation
GCP setup delays: WIF, Cloud KMS, service accounts, and bucket creation require Oppia org admin access. Setup may be delayed if admins are unavailable.	Begin GCP setup documentation and dry-run instructions during Community Bonding. Provide step-by-step setup guides so admins can complete configuration independently. Use a personal GCP project for early development and testing.
Play Console API rate limits or restrictions: The API may throttle requests or have undocumented restrictions on programmatic uploads.	All Play Console API calls include exponential backoff with retry logic. The <code>UploadBinaryToPlayConsole</code> script validates preconditions before attempting any API call (e.g., checking for pending releases, version inversions). The manual fallback via Play Console UI is always available.
LLM changelog quality: Vertex AI may generate inaccurate, overly verbose, or insufficiently descriptive changelogs.	The generated changelog is always proposed as a PR for human review; it is never deployed directly. The PR description includes all raw source material (commit list, fixed issues) so the reviewer can correct the LLM output. Prompt iteration will happen during development to find a reliable prompt template.
Signing key access is irrecoverable: If the Cloud KMS key is accidentally deleted or the key ring is misconfigured, releases cannot be signed.	Cloud KMS keys use the <code>DESTROY_SCHEDULED</code> state with a 24-hour delay before destruction. Key versions can be restored within this window. The key ring and key names are documented. Additionally, the existing manual signing process (local keystore) remains available as a last-resort fallback.

PR review delays: PRs may take longer than expected to review and merge, especially for workflow and infrastructure changes.	Schedule the project to allow for overlapping PR development: start the next PR while the current one is in review. Use the "prerequisite PR" column in the milestone table to identify independent work streams. Submit early PRs during Community Bonding for faster review feedback loops.
Firebase CLI breaking changes: The Firebase CLI is a third-party tool that may change its interface.	Pin the Firebase CLI to a specific version in the workflow. Monitor Firebase release notes during the project. The Firebase App Distribution REST API is available as an alternative if the CLI breaks.
CI flakiness blocking alpha releases: If CI checks are consistently flaky on <code>develop</code> , <code>FindAlphaCandidate</code> will be unable to identify a deployment candidate, causing alpha releases to stall.	<code>FindAlphaCandidate</code> includes robust retry logic: it scans up to 20 recent commits (not just the latest) and reports which specific checks are blocking each candidate. The alpha cron exits cleanly if no candidate is found (no false failures). Persistent CI flakiness is escalated to the team via the workflow's failure notification. The coordinator can always manually trigger <code>build_and_sign.yml</code> with a known-good commit as a workaround.

Key Objectives for Each Milestone

Milestone	Key Objective
Milestone 1	Introduce checked-in changelog support and four manually-triggered GitHub Actions workflows (<code>build_and_sign</code> , <code>deploy_to_firebase</code> , <code>deploy_to_play_console</code> , <code>deploy_updated_changelog</code>) that together enable a release coordinator to build, sign, archive, and deploy any flavor of the app to QA (Firebase) and production (Play Console), and upload changelogs to Play Store, from the GitHub Web UI.
Milestone 2	Automate the remaining release tasks: rewrite release documentation as wiki pages, introduce automated changelog generation with LLM, automated weekly lesson version pinning, and automated weekly alpha deployment via cron.

Implementation Approach

[Android only] Script & CI changes

This section describes the new Kotlin scripts and GitHub Actions workflows, grouped by function.

1. Changelog Infrastructure

New directory: `config/changelogs/`

Changelogs are stored as Markdown files following the naming convention `<major.minor>.md` (e.g., `0.17.md`). Flavor-specific overrides use `<major.minor>_<flavor>.md` (e.g., `0.17_beta.md`). This directory is checked into `develop`, which serves as the source of truth for all changelogs.

The default changelog for a given version applies to all flavors unless an override exists. The deployment scripts look up the changelog by:

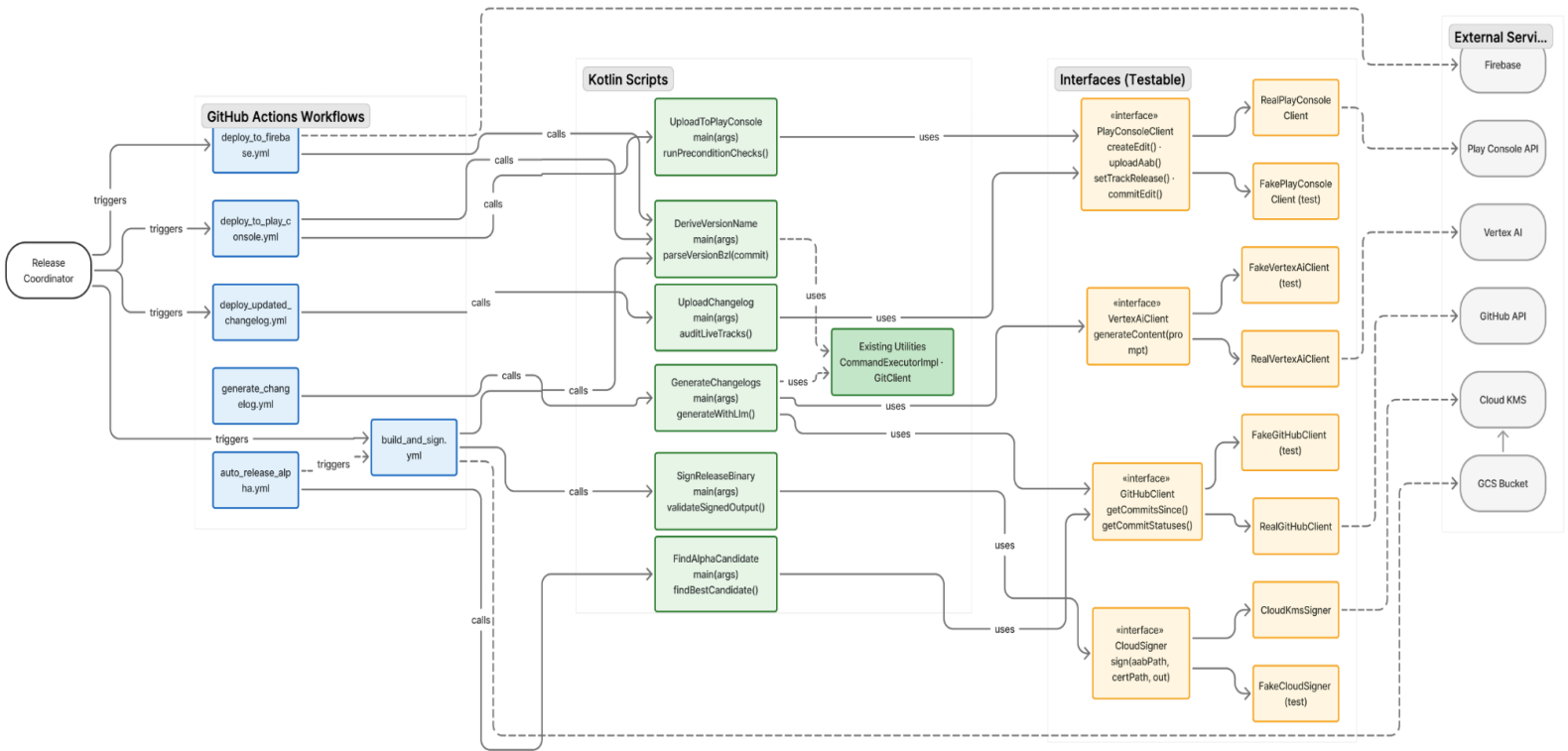
1. Checking for `config/changelogs/<version>_<flavor>.md`.
2. Falling back to `config/changelogs/<version>.md`.
3. Failing if neither exists (a changelog is always required for deployment).

2. New Kotlin Scripts

All scripts are placed under `scripts/src/java/org/oppia/android/scripts/release/` and built as `kt_jvm_binary` Bazel targets. Each script follows the established codebase pattern: a `main()` entry function, argument parsing via `args`, and existing utilities (`CommandExecutorImpl`, `GitClient`). Tests go in the corresponding `javatests/` path using JUnit + Truth assertions.

Script component diagram

The ~~six~~ five new scripts introduce ~~four~~ three abstraction interfaces, each with a real implementation and a test fake. This testability pattern allows every script to be fully unit-tested without live GCP credentials.



Interface	Key methods	Real implementation	Test fake	Used by
CloudSigner	sign(unsignedAabPath, certPath, outputPath)	CloudKmsSigner (jarsigner + KMS)	FakeCloudSigner	SignReleaseBinary
PlayConsoleClient	createEdit(), uploadAab(), setTrackRelease(), commitEdit(), getTrackInfo()	GooglePlayConsoleClient (Play Developer API)	FakePlayConsoleClient	UploadBinaryToPlayConsole, UploadChangelogToPlayConsole
VertexAiClient	generateContent(prompt): String	RealVertexAiClient (Gemini API)	FakeVertexAiClient	GenerateChangelogs
GitHubClient	getIssue(number), getCommitStatuses(sha), getCommitsSince(baseSha)	RealGitHubClient (GitHub REST API)	FakeGitHubClient	GenerateChangelogs, FindAlphaCandidate

All interface files, implementations, and scripts are under `scripts/src/java/org/oppia/android/scripts/release/`. All fakes and tests are under `scripts/src/javatests/org/oppia/android/scripts/release/`.

Workflow → script call graph:

Workflow	Scripts called	Triggers workflows	External services
<code>build_and_sign.yml</code>	<code>SignReleaseBinary</code>	—	Cloud KMS, GCS
<code>deploy_to_firebase.yml</code>	—	—	GCS, Firebase App Distribution
<code>deploy_to_play_console.yml</code>	<code>UploadBinaryToPlayConsole</code>	—	GCS, Play Console API
<code>deploy_updated_changelog.yml</code>	<code>UploadChangelogToPlayConsole</code>	—	Play Console API
<code>generate_changelog.yml</code>	<code>GenerateChangelogs</code>	—	Vertex AI, GitHub API
<code>auto_release_alpha.yml</code>	<code>FindAlphaCandidate</code>	<code>build_and_sign</code> , <code>deploy_to_firebase</code> , <code>deploy_to_play_console</code>	GitHub API

(a) `SignReleaseBinary.kt`

New files: `SignReleaseBinary.kt`, `CloudSigner.kt` (interface), `CloudKmsSigner.kt` (real), `FakeCloudSigner.kt` (test fake), `SignReleaseBinaryTest.kt`

Responsibility: Signs an unsigned AAB using Cloud KMS via Java's built-in `jarsigner` with the Cloud KMS keystore provider. The signing key never leaves the HSM.

Inputs:

- Path to unsigned AAB (produced by `bazel build`)
- Cloud KMS key resource name (e.g., `projects/<id>/locations/global/keyRings/<ring>/cryptoKeys/<key>/cryptoKeyVersions/<ver>`)

- Path to public certificate PEM file (checked into `config/certificate/`)
- Output path for signed AAB

Key behavior:

- Invokes `jarsigner` configured with the Cloud KMS keystore provider, which sends only the AAB digest to KMS and receives the signature back.
- Validates the signed output by checking the signing certificate in `META-INF/`.

Failure cases:

- KMS is unreachable or returns an authentication error → script fails with `CloudKmsAuthenticationException`, logs the WIF identity and requested key resource for debugging.
- Key version is disabled or destroyed → script fails with `KeyVersionUnavailableException`, logs the key version status.
- Certificate in `config/certificate/` doesn't match the KMS key → script fails with `CertificateMismatchException` after signature verification.
- Unsigned AAB file doesn't exist at input path → script fails immediately with `FileNotFoundException`.

Testing: The `CloudSigner` interface abstracts KMS access. Tests use a `FakeCloudSigner` that returns a predictable signature, allowing full coverage without GCP credentials.

Class API:

```
/** Abstracts AAB signing so that tests can avoid real KMS calls. */
interface CloudSigner {
    /** Signs [unsignedAabPath] and writes the signed output to [outputPath]. */
    fun sign(unsignedAabPath: Path, certPath: Path, outputPath: Path)
}

/** Real implementation: invokes jarsigner with the Cloud KMS JCA provider. */
class CloudKmsSigner(
    private val kmsKeyResourceName: String,
    private val commandExecutor: CommandExecutor
) : CloudSigner { ... }

/**
 * The main entrypoint for signing an unsigned AAB using Cloud KMS.
 *
 * Parses command-line arguments, delegates signing to [CloudSigner.sign], and
 * validates
 * the signed output by verifying the certificate in META-INF/. Fails with a
 * non-zero
 * exit code if the KMS key is unavailable, the certificate doesn't match, or the
 * unsigned AAB doesn't exist at the specified path.
 *
 * Usage:
```

```

*   bazel run //scripts:sign_release_binary -- \
*       <unsigned_aab_path> <kms_key_resource_name> <cert_pem_path>
<output_aab_path>
*
* Arguments:
* - unsigned_aab_path: path to the unsigned AAB produced by bazel build.
* - kms_key_resource_name: full KMS key resource name
*   (e.g.
projects/<id>/locations/global/keyRings/<ring>/cryptoKeys/<key>/cryptoKeyVersions/<
ver>).
* - cert_pem_path: path to the public certificate PEM file (checked into
config/certificate/).
* - output_aab_path: destination path for the signed AAB.
*/
fun main(vararg args: String) { ... }

```

(b) TransformAndroidManifest.kt

The version name and version code are computed by `TransformAndroidManifest.kt` and baked directly into the AAB's manifest at Bazel build time.

How it works:

- Version code is computed dynamically from the git commit count relative to `NEW_VERSION_STRATEGY_STARTING_COMMIT_NUMBER`, starting at `BASE_VERSION_CODE`. Each potential release gets 1000 version code slots across 25 flavors/RCs.
- Version name follows the pattern `<major>.<minor>-rc<num>-<flavor>-<10-char-hash>` (e.g. `0.17-rc01-beta-5f3e75afa7`).
- The Bazel `_sign_and_rename_aab` rule (in `oppia_android_application.bzl`) renames the output AAB to `oppia-android-<versionName>.aab` after signing.

Impact on workflows: `deploy_to_firebase.yml` and `deploy_to_play_console.yml` don't need to derive the binary name, so coordinators simply provide the full GCS path directly as a workflow input. `UploadBinaryToPlayConsole` parses the version name and flavor from the AAB filename.

(c) UploadBinaryToPlayConsole.kt

New files: `UploadBinaryToPlayConsole.kt`, `PlayConsoleClient.kt` (interface), `RealPlayConsoleClient.kt` (real), `FakePlayConsoleClient.kt` (test fake), `UploadBinaryToPlayConsoleTest.kt`

Responsibility: Uploads a signed AAB and its changelog to a specified Play Console track. This is the most complex script because it must enforce several safety invariants.

Inputs:

- Path to signed AAB
- Track (`alpha`, `beta`, `production`)

- Rollout percentage (0–100)
- Path to changelog file on `develop`

Key behavior:

- Uses the [Google Play Developer API](#) client library. Creates an Edit → uploads AAB → attaches release notes → sets rollout fraction → commits Edit. [Starting reference Gist](#).
- Play Console API access uses WIF → service account impersonation. The SA must be [invited to Play Console](#) with "Release to production" permission.

Precondition checks (all must pass, otherwise the script fails):

1. A changelog exists for this version on the `develop` branch → otherwise `MissingChangelogException`.
2. No release is already pending for the target track → otherwise `PendingReleaseException` (avoids race condition).
3. The version being deployed is $\geq$ the version currently live on the track (compares `<major>.<minor>`) → otherwise `VersionRegressionException`.
4. Deploying this version would not create a version code inversion (`GA < beta < alpha` must hold across all tracks, accounting for pending releases) → otherwise `VersionCodeInversionException`.

Testing: The `PlayConsoleClient` interface abstracts all API calls. A `FakePlayConsoleClient` simulates track states, pending releases, and version code configurations for testing all precondition branches.

Class API:

```
/** Abstracts Play Developer API so tests can simulate track states. */
interface PlayConsoleClient {
    /**
     * Creates a new edit session for the given [packageName].
     *
     * An edit is required before any other API operations can be staged.
     * Changes do not take effect until [commitEdit] is called.
     *
     * Returns the edit ID string used in subsequent calls.
     */
    fun createEdit(packageName: String): Edit
    /**
     * Uploads the AAB at [aabPath] to the open edit identified by [editId].
     *
     * @param editId the edit session ID returned by [createEdit]
     * @param aabPath the local path to the signed AAB file
     * Returns the [Apk] resource assigned by the Play Developer API.
     */
    fun uploadAab(editId: String, aabPath: Path): Apk
    /**
     * Stages a track release within the edit identified by [editId].
     */
}
```

```

*
* @param editId the edit session ID returned by [createEdit]
* @param track the target Play Console track (e.g. "alpha", "beta", "production")
* @param versionCode the version code of the build being released
* @param rolloutFraction the fraction of users to roll out to (between 0.0 and
1.0)
* @param releaseNotes localized release notes, keyed by BCP-47 Language tag
*/
fun setTrackRelease(
    editId: String, track: String,
    versionCode: Long, rolloutFraction: Double,
    releaseNotes: List<LocalizedText>
)
/**
* Commits the edit identified by [editId], making all staged changes live.
*
* Should only be called after the AAB upload and track release are both staged.
*/
fun commitEdit(editId: String)
/**
* Returns the current [Track] state for the given [track] name.
*
* Used for precondition checks (pending release detection, version inversion
checks)
* before any changes are staged.
*/
fun getTrackInfo(track: String): Track
}
/**
* The main entrypoint for uploading a signed AAB to the Play Console.
*
* Runs precondition checks, then uploads the AAB and changelog to the target
track.
* Fails with a non-zero exit code if any precondition is violated: changelog
missing,
* release already pending, version regression, or version code inversion across
tracks.
*
* Usage:
* bazel run //scripts:upload_binary_to_play_console -- \
*     <path_to_directory_root> <aab_path> <track> <gcp_project_id>
*
* Arguments:
* - path_to_directory_root: directory path to the root of the Oppia Android
repository.
* - aab_path: path to the signed AAB to upload.
* - track: Play Console track to deploy to (alpha/beta/ga).
* - gcp_project_id: GCP project ID for WIF-authenticated Play Console access.

```

```
*/  
fun main(vararg args: String) { ... }
```

(d) UploadChangelogToPlayConsole.kt

New files: UploadChangelogToPlayConsole.kt, UploadChangelogToPlayConsoleTest.kt (reuses PlayConsoleClient interface from (c))

Responsibility: Handles incremental changelog updates after a release is already live (the initial changelog is deployed with the binary by UploadBinaryToPlayConsole; this script only handles corrections).

Inputs: None (reads config/changelogs/ and queries Play Console automatically).

Key behavior:

1. Audits existing Play Console tracks using the Edits API.
2. For each track with a live release, compares the deployed changelog against the latest version in config/changelogs/ on develop.
3. If there's a difference, uploads the updated changelog.

Failure cases:

- A pending release exists for a track → PendingReleaseException (avoids race condition between changelog update and binary deployment).
- The changelog version is newer than the live release → ChangelogVersionMismatchException (the new release hasn't been deployed yet; UploadBinaryToPlayConsole handles initial deployment).
- Play Console API returns an error → retries with exponential backoff up to 3 times, then fails.

(e) GenerateChangelogs.kt

New files: GenerateChangelogs.kt, VertexAiClient.kt (interface), RealVertexAiClient.kt (real), FakeVertexAiClient.kt (test fake), GitHubClient.kt (interface), RealGitHubClient.kt (real), FakeGitHubClient.kt (test fake), GenerateChangelogsTest.kt

Responsibility: When the minor version is bumped in version.bzl, this script auto-generates a user-facing changelog for the *previous* version (i.e., bumping 1.1 → 1.2 generates the changelog for 1.1, since 1.2 is now the unreleased dev build).

Inputs:

- Path to version.bzl (for current version)
- GitHub repository reference (for PR creation)

Key behavior:

1. Reads version.bzl to determine current (new) and previous version numbers.

- Identifies the two most recent `release-*` branches and their merge bases with `develop` (via `GitClient`).
- Collects all merged PR titles and commit messages between those merge bases.
- Parses `Fixes #NNNN` patterns in commit messages, fetches issue titles/descriptions via `GitHubClient`.
- Calls Vertex AI Gemini API with a structured prompt: "Generate 2-3 user-facing sentences describing what's new. Emphasize features first, high priority bug fixes second, ignore dev-only changes." See [reference Gist](#). The prompt template is stored in `scripts/assets/changelog_prompt_template.txt` so it can be tuned without code changes.
- Writes LLM output to `config/changelogs/<previous_version>.md`.
- Creates a PR on branch `automated/changelog-<version>`. The PR description includes: before/after versions, exact commit range, bullet list of all commits, and bullet list of fixed issues.

Failure cases:

- Vertex AI API fails (timeout, quota, error) → **graceful degradation**: script writes the raw commit list to the changelog file with a `<!-- LLM generation failed. Please write a manual summary. -->` marker. The PR is still created; the reviewer writes the summary manually.
- No commits found between release branches → script exits with `EmptyCommitRangeException`.
- GitHub API rate limit hit when fetching issues → retries with backoff, falls back to issue numbers without descriptions.

Testing: `VertexAiClient` interface with `FakeVertexAiClient` that returns canned responses. Tests cover: successful generation, LLM failure fallback, empty commit range, and issue-parsing logic.

Class API:

```
/** Abstracts Vertex AI so tests can return canned LLM responses. */
interface VertexAiClient {
    /** Sends [prompt] to Gemini and returns the generated text. */
    fun generateContent(prompt: String): String
}

/** Abstracts GitHub REST API for commit/issue queries. */
interface GitHubClient {
    /**
     * Returns the List of commits on [headRef] since [baseSha], in chronological
     * order.
     *
     * @param baseSha the SHA of the last commit on the previous release branch
     * @param headRef the git ref of the current release branch (e.g. release-0.17)
     */
    fun getCommitsSince(baseSha: String, headRef: String): List<Commit>

    /**
     * Returns the GitHub [Issue] for the given issue [number].
     *
     * Used to fetch issue titles for changelog entries referencing fixed bugs.
     */
}
```

```

    */
fun getIssue(number: Int): Issue
/**
 * Returns the list of CI [CommitStatus]es for the given commit [sha].
 *
 * Used by [FindAlphaCandidate] to determine whether a commit has passing CI.
 */
fun getCommitStatuses(sha: String): List<CommitStatus>
}
/**
 * The main entrypoint for automated changelog generation.
 *
 * Collects commits between release branches, calls Vertex AI for a user-facing
 * changelog summary, writes the changelog file, and opens a GitHub PR with all
 * reference material. If the LLM call fails, the PR is still created with the
 * raw commit list so the coordinator can write the summary manually.
 *
 * Usage:
 *   bazel run //scripts:generate_changelogs -- \
 *     <path_to_directory_root> <base_ref> <head_ref> <version> <gcp_project_id>
 *
 * Arguments:
 * - path_to_directory_root: directory path to the root of the Oppia Android
repository.
 * - base_ref: the git ref of the previous release branch (e.g. release-0.16).
 * - head_ref: the git ref of the current release branch (e.g. release-0.17).
 * - version: the release version string (e.g. 0.17).
 * - gcp_project_id: GCP project ID for WIF-authenticated Vertex AI access.
 */
fun main(vararg args: String) { ... }

```

(f) FindAlphaCandidate.kt

New files: FindAlphaCandidate.kt, FindAlphaCandidateTest.kt (reuses GitHubClient interface from (e))

Responsibility: Identifies the best commit on `develop` for the next alpha deployment: the newest commit with all CI checks passing.

Inputs:

- GitHub repository reference
- `latest-alpha` tag (baseline to compare against)

Output: A commit SHA, or an exit code indicating no candidate.

Key behavior:

1. Queries the GitHub API for all commits on `develop` since `latest-alpha`.
2. For each commit (newest first), queries its combined CI status via the GitHub Checks API.
3. Returns the first commit where all required checks are passing.

Failure cases:

- No new commits since `latest-alpha` → exits cleanly with exit code 0 and log message "No new commits since latest-alpha." The cron interprets this as "nothing to do."
- New commits exist but none have all CI checks passing → exits with exit code 1 and logs each commit with its failing check names. This is the CI flakiness scenario so the cron marks the run as failed, alerting the team.
- GitHub API is unreachable → retries with backoff up to 3 times, then exits with error.

3. New GitHub Actions Workflows

All workflows use the existing [set-up-android-bazel-build-environment](#) composite action for Bazel setup and caching. All release workflows specify `environment: oppia-android-release-env` to enforce required reviewer approval before execution. Because build and deploy are separate workflows (see Decision 4), the coordinator can re-run any failed deployment without rebuilding.

(a) `build_and_sign.yml` — Build, sign, and archive a release binary.

Aspect	Detail
Trigger	<code>workflow_dispatch</code> (manual) and <code>workflow_call</code> (reusable, called by <code>auto_release_alpha</code>)
Inputs	<code>flavor</code> (alpha/beta/ga), <code>source_ref</code> (release branch or <code>latest-alpha</code> tag)
Environment	<code>oppia-android-release-env</code> (requires reviewer approval)

Steps:

1. **Validate inputs:** reject invalid flavors, verify `source_ref` exists via `git ls-remote`.
2. Checkout the source ref at its latest commit.
3. Set up Bazel build environment via the existing composite action.
4. Build AAB: `bazel build --compilation_mode=opt //:oppia_<flavor>`.
5. Authenticate to GCP via WIF (`google-github-actions/auth`): short-lived credentials, no stored keys.
6. Run `SignReleaseBinary`: AAB digest goes to Cloud KMS, signature comes back, key never leaves HSM.
7. Upload signed AAB to the flavor-specific GCS bucket. **Fails** if the binary already exists (preventing overwrites ensures a given RC is immutable once archived)

(b) `deploy_to_firebase.yml` — Distribute a QA build to testers.

Aspect	Detail
Trigger	<code>workflow_dispatch</code> and <code>workflow_call</code>

Inputs	<code>gcs_aab_path</code> (full GCS URI to signed AAB) , <code>release_notes</code> (optional), <code>tester_groups</code> (optional defaults to qa-team)
Environment	<code>oppia-android-release-env</code>

Steps:

1. Validate `gcs_aab_path` format (must match `gs://oppia-android-{alpha|beta|ga}-releases/...aab`); extract flavor from path..
2. Authenticate to GCP via WIF.
3. Download the signed AAB from GCS via `gsutil cp`. Fails if the binary is not found (build must happen first).
4. Install Node.js and Firebase CLI (`npm install -g firebase-tools`).
5. Resolve Firebase App ID for the flavor from GitHub secrets.
6. Upload via `firebase appdistribution:distribute <aab> --app <app_id> --groups <tester_groups> --release-notes "<notes>"`. Testers receive a direct download link via email, hence no Play Store sync delay.

(c) `deploy_to_play_console.yml` – Deploy a release to production users.

Aspect	Detail
Trigger	<code>workflow_dispatch</code> and <code>workflow_call</code>
Inputs	<code>gcs_aab_path</code> (full GCS URI to signed AAB), <code>track</code> (alpha/beta/production), <code>rollout_fraction</code> (0.0-1.0)
Environment	<code>oppia-android-release-env</code>

Steps:

1. **Validate inputs:** especially that the flavor-track mapping is correct (e.g., beta flavor → beta track).
2. Authenticate to GCP via WIF.
3. Download the signed AAB from GCS via `gsutil cp`: the same immutable binary that QA tested via Firebase.
4. Run `UploadBinaryToPlayConsole`:
 - a. pre-upload checks – pending release detection, changelog existence in `config/changelogs/<major.minor>[_<flavor>].md`;
 - b. upload AAB;
 - c. version inversion check using the returned version code;
 - d. set track release;
 - e. commit edit.

(d) `deploy_updated_changelog.yml` – Push changelog corrections to live releases.

Aspect	Detail
Trigger	<code>on: push</code> with <code>paths: ['config/changelogs/**']</code> on <code>develop</code> , plus <code>workflow_dispatch</code>

Environment	<code>oppia-android-release-env</code>
--------------------	--

Steps:

1. Authenticate to GCP via WIF.
2. Run `UploadChangelogToPlayConsole`: audits live tracks, detects changelog diffs, uploads updates. Fails safely if a pending release exists on any track.

(e) `generate_changelog.yml` – Auto-generate changelogs when the version is bumped.

Aspect	Detail
Trigger	<code>on: push</code> with <code>paths: ['version.bzl']</code> on <code>develop</code> , plus <code>workflow_dispatch</code>
Environment	<code>oppia-android-release-env</code> (for Vertex AI access)

Steps:

1. Authenticate to GCP via WIF for Vertex AI access.
2. Run `GenerateChangelogs`: generates the changelog, creates a PR with all reference material. If the LLM call fails, the PR is still created with raw data for manual editing.

(f) `pull_latest_lesson_versions.yml` – Weekly lesson version pinning.

Aspect	Detail
Trigger	<code>schedule: cron '0 10 * * 1'</code> (weekly on Monday 10:00 UTC), plus <code>workflow_dispatch</code>

Steps:

1. Checkout `develop`.
2. Run `//scripts:download_lesson_list` to regenerate `config/pinned_download_list_versions.textproto`.
3. If there are changes, force-push to the dedicated branch `automated/lesson-versions` (same branch reused each week, like the Translatewiki model).
4. Create or update the PR using `peter-evans/create-pull-request` action.

(g) `auto_release_alpha.yml` – Weekly automated alpha deployment.

Aspect	Detail
Trigger	<code>schedule: cron '0 12 * * 1'</code> (weekly on Monday 12:00 UTC), plus <code>workflow_dispatch</code>
Environment	<code>oppia-android-release-env</code>

Steps:

1. Run `FindAlphaCandidate`: finds latest `develop` commit with passing CI. If no candidate exists, the workflow exits cleanly (no new alpha this week).
2. Update `latest-alpha` tag: `git tag -f latest-alpha <commit> && git push -f origin latest-alpha`.
3. Trigger `build_and_sign.yml` for alpha via `workflow_call`: builds, signs, archives.
4. Trigger `deploy_to_firebase.yml`: QA team gets the alpha build immediately.
5. Trigger `deploy_to_play_console.yml`: alpha users on Play Store get the update at 100% rollout.
6. Output summary: selected commit, skipped commits with reasons, deployment status.

4. Environment and Infrastructure Setup

The project requires infrastructure across four services: Google Cloud Platform (GCP), GitHub, Firebase, and Google Play Console.

GCP Infrastructure

Resource	Purpose	Configuration Details
GCP Project	Hosts all cloud resources	New project (or existing Oppia project)
Workload Identity Federation (WIF)	Keyless auth from GitHub Actions → GCP	Identity pool with attribute conditions: <code>attribute.repository == "oppia/oppia-android"</code> , <code>attribute.environment == "oppia-android-release-env"</code> , <code>attribute.ref</code> matches <code>refs/heads/release-*</code> or <code>refs/tags/latest-alpha</code>
Service Accounts	Authenticate to individual GCP services	Separate SAs for: signing (KMS), archiving (GCS), Play Console API, Firebase, Vertex AI
Cloud KMS Key Ring	HSM-backed asymmetric signing key	Key ring with a single signing key; <code>DESTROY_SCHEDULED</code> state with 24-hour delay
GCS Buckets (×3)	Archive signed release binaries	<code>oppia-android-alpha-releases</code> (90-day auto-delete), <code>oppia-android-beta-releases</code> (90-day archive, 365-day delete), <code>oppia-android-prod-releases</code> (90-day archive, never delete)
Vertex AI (Gemini API)	LLM-powered changelog generation	API enabled in the GCP project; accessed via WIF credentials

Github Infrastructure

Resource	Purpose	Configuration Details
GitHub Environment: <code>oppia-android-release-env</code>	Gate release workflows behind manual approval	Required reviewers (org admins); deployment branches restricted to <code>release-*</code> and tag <code>latest-alpha</code>
Environment Secret: GCP Project ID	Identify the GCP project for WIF	Stored as an environment secret

Firebase Infrastructure

Resource	Purpose	Configuration Details
Firebase Project	Host App Distribution for QA builds	Linked to the GCP project
App Distribution	Distribute signed AABs to testers	App registered, testers managed via Google Group
Tester Group: <code>qa-team</code>	Target QA testers for build distribution	Managed in Firebase console (not in codebase)

Play Console Infrastructure

Resource	Purpose	Configuration Details
Play Console API Access	Programmatic AAB upload + changelog management	Service account invited to Play Console with "Release to production" permission

Personal Development Environment (Avoids Maintainer Dependency)

To ensure development is **never blocked** by org admin availability, a **personal GCP project** will be set up during Community Bonding that mirrors the production infrastructure:

Personal Resource	Mirrors	Purpose
Personal GCP Project	Oppia GCP Project	Hosts all test resources

Personal WIF Identity Pool	Production WIF	Allows keyless auth from my fork's GitHub Actions
Personal Cloud KMS Key Ring	Production KMS	Test signing key for development
Personal GCS Buckets (×3)	Production GCS Buckets	Test binary archiving
Personal Firebase Project	Oppia Firebase	Test App Distribution with personal tester group
Personal Vertex AI API Access	Production Vertex AI	Test changelog generation
Test App on Play Console	Oppia's real app listing	Test AAB uploads and changelog management

Why this works: Since every script uses interface abstractions, the **same code** runs against both the personal test environment and Oppia's production environment. The only difference is configuration (GCP project ID, KMS key resource name, GCS bucket names, Firebase app ID, Play Console package name), which is passed as workflow inputs or environment variables, hence never hardcoded.

Detailed Setup Timeline

#	Setup Item	When	Who Sets It Up	Can Develop Without It?
1	Personal GCP Project (WIF, KMS, GCS, Vertex AI)	Community Bonding (before May 25)	Me (fully independent)	N/A (this IS the dev environment)
2	Personal Firebase Project + App Distribution	Community Bonding	Me (fully independent)	N/A
3	Personal fork with test workflows	Community Bonding	Me (fully independent)	N/A
4	GCP setup instructions document for org admins	By May 27 (M1 Row 1)	Me (delivered to admins)	N/A

5	Oppia GCP Project (WIF, KMS, GCS, service accounts)	After receiving instructions (target: early June)	Org admins (following my step-by-step guide)	Yes (use personal GCP project)
6	GitHub Environment: <code>oppia-android-release-env</code>	Community Bonding or Early M1	Org admins	Yes (tested without environment gate on personal fork)
7	Oppia Firebase Project + App Distribution	Before M1 Row 6 (target: mid-June)	Org admins	Yes (use personal Firebase project)
8	Play Console API access (SA invited)	Before M1 Row 7 (target: mid-June)	Org admins	Partially (unit tests use <code>FakePlayConsoleClient</code> ; integration tests may use personal test app or be deferred)
9	Vertex AI API enabled	Before M2 Row 1 (target: mid-July)	Org admins	Yes (use personal GCP project's Vertex AI)

Permissions Required

Service	Permission / Resource	Purpose	Granted By	When Needed
GitHub	Write access to <code>oppia-android</code> repo	Submit PRs for scripts, workflows, and wiki pages	Already available (collaborator access)	Already available
GitHub	Create <code>oppia-android-release-env</code> environment	Gate release workflows behind reviewer approval	Org admin	Community Bonding
GitHub	Required reviewer role on environment	Approve workflow runs	Org admin	Community Bonding
GitHub	Environment secret: GCP Project ID	Configure WIF authentication	Org admin	Community Bonding / Early M1

GCP	<code>roles/iam.workloadIdentityPoolAdmin</code>	Set up keyless auth for GitHub Actions	Org admin (I provide the config)	By May 27 (M1 Row 1)
GCP	<code>roles/cloudkms.admin</code>	Create the HSM-backed signing key	Org admin (I provide the config)	By May 27
GCP	<code>roles/storage.admin</code>	Create 3 flavor-specific GCS buckets	Org admin (I provide the config)	By May 27
GCP	Service accounts with appropriate IAM roles	Signing (KMS), Archiving (GCS), Play Console, Firebase, Vertex AI	Org admin (I provide the config)	By May 27
GCP	Vertex AI Gemini API enabled	Allow changelog generation LLM calls	Org admin	Before M2 Row 1
Firebase	Firebase project with App Distribution enabled	Distribute QA builds	Org admin	Before M1 Row 6
Firebase	Tester group (<code>qa-team</code>) with tester emails	Target testers for build distribution	Org admin	Before M1 Row 6
Play Console	Service account invited with "Release to production"	Programmatic AAB upload, changelog management, track queries	Org admin	Before M1 Row 7

Documentation changes

Four new wiki pages are created under a new "Releasing" section of the Oppia Android wiki:

1. **App & Feature Release Process:** High-level overview showing the lifecycle of both binary releases (alpha → beta → GA) and feature releases (feature flag → alpha testing → product review → QA → team lead sign-off → production enable → cleanup). Includes diagrams showing how changes flow from `develop` to users and how feature flags interact with release branches.

2. **Release Playbook:** Step-by-step coordinator guide built around the new automation. Each step references the corresponding GitHub Actions workflow with exact instructions ("Navigate to Actions → [build_and_sign](#), select flavor: beta, source ref: release-0.17, click Run workflow"). Also covers the weekly coordinator checklist (check alpha approvals, review changelog PRs, review lesson version PRs, review TranslateWiki PRs). Accounts for the [#5033](#) version code strategy.
3. **In-Depth Release Reference:** Manual fallback guide for every automated step. This is the "if GitHub Actions is broken" reference. Covers: manual branch cutting, manual Bazel build + optimization, local signing with [apksigner](#), manual Play Console upload, manual Firebase upload. Links from the Playbook for easy jump access.
4. **Feature Development Guide:** A developer-facing guide covering how to gate new features behind feature flags, how those features progress through the release lifecycle (alpha → beta → GA), and how to request production enablement once testing is complete. Linked from the App & Feature Release Process page.

Metrics Plan

This is an infrastructure project with no app-level user-facing events. There are no new analytics events to add to the app. The primary "metrics" are operational:

Event	Event parameters	Do we already record the event + parameters? • If so, please link to the corresponding code on GitHub. • If not, describe the changes needed to do so.
Not applicable	Not applicable	This project introduces no changes to the Android app's runtime behavior. All changes are CI/CD scripts, GitHub Actions workflows, and wiki documentation. There are no new user-facing events or analytics to record.

Implementation Plan

Milestone Table

Milestone 1

Key objective for this milestone:

Introduce checked-in changelog support and four manually-triggered GitHub Actions workflows ([build_and_sign](#), [deploy_to_firebase](#), [deploy_to_play_console](#), [deploy_updated_changelog](#)) that enable a release coordinator to build, sign, archive, and deploy any flavor of the app to QA (Firebase) and production (Play Console) from the GitHub Web UI.

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge

1	[Action] Finalize and deliver GCP setup instructions for org admins: WIF identity pool, service accounts, Cloud KMS key ring, GCS buckets, GitHub environment. Provide to admins for setup.	None	Completion: May 27	–
2	[PR] Add <code>config/changelogs/</code> directory with initial changelog format, README, and a sample <code>0.16.md</code> . Add <code>config/certificate/</code> with placeholder for public certificate.	None	May 25	May 30
3	[PR] Implement <code>DeriveVersionNameForCommit.kt</code> script with unit tests. Reads <code>version.bzl</code> for a given commit and constructs the fully qualified binary name.	None	May 26	Jun 1
4	[PR] Implement <code>SignReleaseBinary.kt</code> script with unit tests. Abstracts Cloud KMS signing behind an interface (<code>CloudSigner</code>) with a real implementation (<code>jarsigner</code> + KMS provider) and a fake for testing.	None	May 28	Jun 4
5	[PR] ★ Implement <code>build_and_sign.yml</code> workflow. Validates inputs, builds AAB with Bazel, signs via <code>SignReleaseBinary</code> , uploads to GCS. Includes the GitHub environment configuration.	2, 3, 4	Jun 1	Jun 15
–	At this point: A release coordinator can build and sign any flavor of the app from the GitHub UI. The signed binary is archived in GCS. Acceptance tests for User Story 2 can be verified.			
6	[PR] Implement <code>deploy_to_firebase.yml</code> workflow. Downloads signed AAB from GCS, uploads to Firebase App Distribution via Firebase CLI.	5	Jun 4	Jun 17
7	[PR] ★ Implement <code>UploadBinaryToPlayConsole.kt</code> script with unit tests. Abstracts Play Console API behind a <code>PlayConsoleClient</code> interface with real and test implementations. Includes precondition checks (version inversion, pending release detection, changelog existence). (Most complex script; submitted early for review.)	3	Jun 4	Jun 19

8	[PR] Implement <code>deploy_to_play_console.yml</code> workflow. Downloads signed AAB from GCS, runs <code>UploadBinaryToPlayConsole</code> .	5, 7	Jun 9	Jun 22
9	[PR] Implement <code>UploadChangelogToPlayConsole.kt</code> script with unit tests. Audits live tracks, detects changelog diffs, uploads updates.	7	Jun 11	Jun 23
10	[PR] Implement <code>deploy_updated_changelog.yml</code> workflow. Triggered on changelog push, runs <code>UploadChangelogToPlayConsole</code> .	9	Jun 15	Jun 26
–	At this point: The full M1 build-sign-deploy pipeline is operational. All four workflows are functional. Acceptance tests for User Stories 1-4, 7, and 9 can be verified.			
11 (PM Demo)	Demonstrate the complete workflow: build+sign a beta release, deploy to Firebase (show QA receiving the build), deploy to Play Console (show track update), update a changelog (show Play Console listing update). Demonstrate the complete M1 pipeline (≤ 10 min). Pre-built before the demo: a signed beta AAB is already archived in GCS. Live during the demo: (1) trigger <code>deploy_to_firebase.yml</code> from the GitHub UI and show the Firebase App Distribution email arriving in the QA tester inbox; (2) trigger <code>deploy_to_play_console.yml</code> and show the Play Console track updating; (3) show <code>deploy_updated_changelog.yml</code> output. Walk through the GCS archive and workflow run logs.	6, 8, 10	Jun 29	
12	[PR] End-to-end acceptance test documentation and any remaining unit test coverage. Write test cases from the Testing Plan for User Stories 1-4, 7, 9. Acceptance test documentation for User	10	Jun 20	Jul 3

	Stories 1–4, 7, 9. These are manual critical user journey validations (not automated tests): a Markdown document recording the test procedure, expected outcome, and actual result for each User Story, run against the live infrastructure. Also resolves any remaining unit test gaps. (PR drafting starts Jun 20 in parallel with Row 10 review; actual results filled in after Row 10 merges Jun 26.)			
--	---	--	--	--

Milestone 2

Key objective for this milestone:

Automate the remaining release tasks: rewrite release documentation as wiki pages, introduce automated changelog generation with LLM, automated weekly lesson version pinning, and automated weekly alpha deployment cron.

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
1	[PR] ★ Implement <code>GenerateChangeLogs.kt</code> script with unit tests. Collects commits and issues between release branches, calls Vertex AI API, generates changelog file, creates PR. Includes LLM fallback logic.	M1 complete	Jul 18	Aug 4
2	[PR] Implement <code>generate_changelog.yml</code> workflow. Triggered on <code>version.bzl</code> push, runs <code>GenerateChangeLogs</code> .	1	Jul 24	Aug 7
-	At this point: Changelog generation is automated. Acceptance tests for User Story 8 can be verified.			
3 (Mentor Demo)	Demonstrate changelog generation: bump <code>version.bzl</code> on a test branch, show the workflow creating a PR with LLM-generated changelog and reference material. Show the LLM fallback behavior.	2	Aug 8	
4	[PR] Implement <code>pull_latest_lesson_versions.yml</code> cron workflow. Runs	None	Jul 28	Aug 4

	<code>download_lesson_list</code> , creates/updates PR on dedicated branch.			
5	[PR] Implement <code>FindAlphaCandidate.kt</code> script with unit tests. Queries GitHub API for CI statuses, identifies deployment candidate.	None	Jul 28	Aug 5
6	[PR] Implement <code>auto_release_alpha.yml</code> cron workflow. Runs <code>FindAlphaCandidate</code> , updates <code>latest-alpha</code> tag, triggers build+sign+deploy workflow chain.	5, M1.5	Aug 2	Aug 14
-	At this point: All automation is in place: changelog generation, lesson pinning, and alpha deployment crons. Acceptance tests for User Stories 5, 6, 8, 10 can be verified.			
7 (PM Mentor Demo)	Demonstrate alpha cron: show candidate selection, tag update, and automated deployment to Firebase + Play Console. Demonstrate lesson pinning: show weekly PR creation.	4, 6	Aug 15	
8	[PR] Write "App & Feature Release Process" wiki page: binary + feature lifecycle overview with diagrams.	M1 complete	Aug 4	Aug 12
9	[PR] Write "Release Playbook" wiki page: step-by-step coordinator guide using new automation. Includes weekly checklist.	8	Aug 8	Aug 14
10	[PR] Write "In-Depth Release Reference" wiki page: manual fallback for every automated step. Linked from Playbook.	9	Aug 10	Aug 16
11	[PR] Write "Feature Development Guide" wiki page: how developers gate features behind flags, navigate the release process, and request production enablement. Linked from App & Feature Release Process.	8	Aug 12	Aug 17
12	[PR] Final acceptance test coverage and end-to-end validation. Write remaining test cases from Testing Plan for User Stories 5, 6, 8, 10, 11.	6, 11	Aug 14	Aug 21

Rest of the days not included in the table will be used as a buffer to follow up any pending tasks, address review feedback, or handle unexpected delays.

Feature Flags

This project introduces no feature flags. All changes are CI/CD infrastructure.

Future Work

Once the core automation is in place, the next natural steps would be fully automating beta and GA deployments via cron (currently only alpha is automated), and integrating changelog translations through TranslateWiki so localized release notes ship without manual effort. Longer term, a release health dashboard pulling crash metrics and ANR rates from Play Console could inform automated rollout/halt decisions.